Saint Petersburg State University

GRIDIAEVA Polina Aleksandrovna

Final Qualification Thesis

Machine Learning Methods in the Study of Energy Characteristics of Single Events in a Collider Experiment

Level of education: Bachelor's degree
Field of study 03.03.01 "Applied Mathematics and Physics"
Main educational programme SV.5009.2021 "Applied Physics and Mathematics"

Scientific supervisor:
Professor, Department of Computational Physics, Doctor of Physical and Mathematical Sciences, Rudnev Vladimir Aleksandrovich

Reviewer: Professor, Department of Nuclear Physics Research Methods, Doctor of Physical and Mathematical Sciences, Valiev Farkhat Fagimovich

Saint Petersburg
2025

Contents

Introduction **3**

1. Artificial Neural Networks **5**

2. Model of the Experiment **7**

3. Implementation **8**

4. Verification of the Fulfilment of the ECL and MCL **13**

5. Energy and Momentum Filters **15**

6. Construction and Analysis of the Dependence of the Interaction Energy on the Impact Parameter **15**

7. Construction of a Function Mapping the Signal from the Detector to the Released Energy **20**

Conclusions and Further Directions of Research **31**

Equipment Used **32**

References **33**

Appendix 1 . . . . . **34**

Appendix 2 . . . . . **35**

Introduction

High-energy physics is a field of science that studies the fundamental interactions of elementary particles and atomic nuclei at energies significantly exceeding their rest energies [1]. These studies make it possible not only to look into the nature of matter at the microscopic level, but also to reproduce the conditions that existed in the Universe in the first moments after the Big Bang. Modern theoretical models, such as the Standard Model and its extensions, require experimental confirmation, which makes high-energy physics one of the key directions of fundamental science. Experimental physics in this field is based on the use of particle accelerators, which accelerate charged particles to velocities close to the speed of light and collide them under certain conditions. The most important objects of research are protons, heavy ions, as well as antiprotons and positrons, whose interactions make it possible to study the quark-gluon plasma, the mechanisms of symmetry breaking and the properties of new particles. When analysing the results of collisions, special attention is paid to the process of registration and interpretation of data. Not all interaction parameters can be measured directly. For example, such important characteristics as the impact parameter, which determines the degree of centrality of the collision, or the coordinate of the interaction along the beam axis, are calculated by indirect methods. Their accurate determination plays a key role in the study of many-particle interactions and the verification of theoretical models [2]. In addition, data processing in high-energy physics requires the application of modern methods for the analysis of large volumes of information. For this purpose, machine learning algorithms and statistical methods are used, which make it possible to efficiently filter experimental events, to single out rare processes and to model the dynamics of particle interactions. The optimisation of these methods contributes to a more accurate reconstruction of collision parameters and to the search for new physical phenomena. The application of various data analysis methods is of critical importance in expanding our understanding of the fundamental laws of nature.

In the collision model considered in this work, nuclei of heavy ions interact in the collider, and in the course of this process a large number of secondary particles is produced, propagating in different directions from the interaction zone. Using the parameters registered by microchannel plate detectors (for example, time of flight, coordinates of the collision of a particle with the detector, detector number), it is possible to build a physical model of the event, estimating the main characteristics of the collision, including the energy released during the collision.

One of the effective approaches to solving regression problems in the field of nuclear collision analysis is the application of deep neural networks. In particular, in work [3] a model of a deep fully connected neural network was proposed, capable of predicting the elliptic flow coefficient on the basis of event parameters, such as centrality and transverse momentum of particles. In another work [4] the authors applied a convolutional neural network to reconstruct physical quantities, including energy, from the spatial distribution of energy density in ultrarelativistic heavy-ion collisions. Both approaches demonstrate high accuracy and justify the relevance of using neural network methods in problems of predicting the characteristics of collisions.

The aim of the present work is the event-level estimation of the energy of the particles that interacted during the collision of two gold nuclei, using machine learning technologies, on the basis of data obtained from detectors on microchannel plates, in order to subsequently install these detectors at MPD NICA [4]. The QGSM generator is used as the model of collision dynamics.

To achieve this aim, the following tasks were set:

1. To check the fulfilment of the laws of conservation of energy and momentum for the QGSM generator in a head-on collision of gold nuclei (Au-Au) with initial energy $\sqrt{s_{NN}} = 11.006$ GeV per nucleon-nucleon pair.

2. Taking into account the estimate of the CL, to formulate the criteria by which the energy released during the collision must be determined.

3. To construct plots of the dependence of the energy released during the interaction on the impact parameter of the collision and to explain their physical meaning.

4. To construct a function capable of mapping the signal from the microchannel plate detectors to the energy released in the collision.

1. Artificial Neural Networks

An artificial neural network (ANN) is a mathematical model inspired by the work of a real biological neural network: the brain receives information from the outside world by means of all the sense organs, and then this information is transmitted between nodes connected to each other – neurons. A neural network is a function that transforms the input data, processing them by means of one or several hidden layers, and passes the transformed data to the output. Each neuron in an ANN performs a number of simple transformations: weighted summation of inputs, addition of a bias, application of an activation function. The output data from one layer of an ANN can be described by the following formula:

$$y = \sigma(W \cdot x + b),$$

where $x \in \mathbb{R}^n$ is the input vector, $W \in \mathbb{R}^{m \times n}$ is the weight matrix, $b \in \mathbb{R}^n$ is the bias vector, $\sigma(\cdot)$ is the activation function.

In this work two types of neural networks were considered: fully connected and hybrid (with the addition of a convolutional layer). A network is considered fully connected if in a layer each neuron is connected to all neurons of the previous layer. Fully connected neural networks are a universal approximant, capable of approximating arbitrary continuous functions with a given accuracy. This makes them an effective tool for solving regression problems and, as a consequence, their use in this work is justified. In the case of a convolutional neural network, instead of full connectivity, a convolution of small trainable filters with the input data is applied. Formally, the output of a convolutional layer is given by the following formula:

$$out(i,j) = \sum_{m=0}^{k-1} \sum_{n=0}^{l-1} input(i+m,\, j+n) \cdot kernel(m,n), \quad \text{where}$$

$k \times l$ is the size of the convolution kernel, *input* is the input features, *out* is the output data, *kernel* is the filter that is applied to the input data.

Convolutional neural networks (CNN) have proven themselves as an effective tool in computer vision tasks and are widely used in image analysis. The data coming from the detectors can be represented in the form of a two-dimensional array, similarly to images, therefore the use of convolutional layers is also logical and justified.

The SiLU function (Sigmoid Linear Unit) was used as the activation function:

$$SiLU(x) = x \cdot \sigma(x) = \frac{x}{1+e^{-x}},$$

where $\sigma(x)$ is the sigmoid activation function.

Unlike other activation functions, such as ReLU, SiLU is smooth and differentiable on the whole number line, which contributes to improving the quality of model training. The activation function is needed in order to add non-linearity to the model. This helps to extract complex dependencies in the data under study.

During the training of the model, the optimisation of the training parameters takes place: neurons in the layers, weights and biases. The goal of training is to minimise the error function, which reflects the degree of discrepancy between the values predicted by the model and the true values. In this work the mean squared error (MSE) is used as the loss function:

$$\mathscr{L}_{MSE} = \frac{1}{N} \sum_{i=1}^{N} (\hat{y}_i - y_i)^2, \quad \text{where}$$

$\hat{y}_i$ is the predicted value, and y_i is the true value, N is the total number of observations (examples in the dataset for training the NN).

Thanks to the loss function, it is possible to evaluate how good or bad the result produced by the model under consideration is. Minimisation of this function makes it possible to increase the accuracy of the model and to ensure its ability to find and generalise the regularities identified on the training dataset.

To optimise the model parameters, this work uses the *Adam* optimiser (Adaptive Momentum Estimation). *Adam* automatically adjusts the gradient descent step at each stage of training, which increases the stability of the model and also reduces the probability of "getting stuck" in local minima. It is precisely thanks to the optimiser that it is possible to achieve more stable and reproducible results when training the neural network.

Model training in ML can be of several types. The most common are supervised, unsupervised and reinforcement learning. This work uses supervised learning – a type of machine learning in which the model is trained on labelled data, that is, on pairs of input features and the corresponding target values. Subsequently the model tries to make predictions for new, previously unseen data, based on the ability to generalise dependencies and find regularities acquired on the training sample. Within the framework of this work supervised learning is used, since one of the tasks is the prediction of numerical values – the energy released during a nuclear collision – from known characteristics of events.

As already mentioned earlier, to evaluate the quality of the model and to prevent overfitting, the initial dataset can be divided into training and test samples. In this work the division is made in the ratio 80/20. That is, 80% of the data are used for training the model – optimisation of its internal parameters (weights, biases), and 20% – to check how well the model copes with the task on new data never seen by it before. Splitting the sample is necessary in order to evaluate the generalisation ability of the model, and not only its ability to memorise the training data. If the model copes poorly with the task on the test sample, then one can speak of overfitting – a situation when the model has adjusted too precisely to the training sample and has lost the ability to generalise.

In the context of this work, the model learns to predict the energy released during the collision of two gold nuclei, using the characteristics of events. This is the so-called regression problem – a type of problem in machine learning in which the model is trained to predict continuous values on the basis of the input data.

2. Model of the Experiment

In this experiment, collisions of two gold nuclei Au+Au ($A = 197$ – atomic mass, $Z = 79$ – charge of the gold nucleus) at high energies are considered. At the moment of collision, energy is released, accompanied by the production of secondary particles, which subsequently fly apart in all directions and are registered by the detector system. The collisions in this work are single, and are also considered in the centre-of-mass system. For each event the detector registers the time of flight and the coordinates of the collision of the particle with the surface of the detector. The registered information makes it possible to judge the spatio-temporal structure of the event, and from the totality of the obtained signals it is possible to reconstruct the physical characteristics of the collision itself – in particular, the energy released as a result of the collision. Such an approach underlies many data analysis tasks in high-energy physics: the released energy is not observed directly, but is determined on the basis of the information obtained from the detectors.

For the preparation of the experimental data, as well as for the further analysis of the energy characteristics of single events in a collider experiment, the configuration of the detector system is very important. In this work the detectors on microchannel plates are located at the same distance from the point of collision of the gold nuclei, equal to $z_1 = z_2 = z = 400$ cm. In addition, the detectors are placed inside the accelerator and have a ring shape: the diameter of the outer ring is $D = 50$ cm, the diameter of the inner ring is $d = 5$ cm. Moreover, the detectors are divided into sensitive plates (pads) by angle and by radius. Each pad is capable of registering information about the particle that hit it, independently of the other sensitive plates. It should be noted that each pad is capable of registering only the particle that reached it first. We divide the detectors into 60 pads: we take 6 rings and 10 sectors.

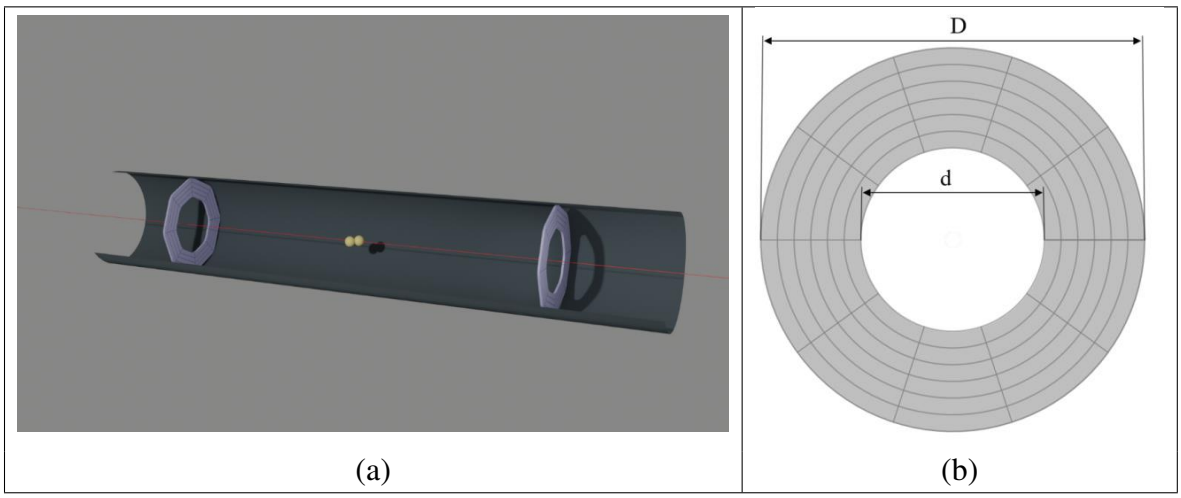


Fig. 1: Scheme of the components of the detector setup: a – scheme of the arrangement of the detectors relative to the point of collision of the nuclei, b – division into sensitive pads by angle and radius.

Let us now present some of the formulas and constants actively used in the course of the work. For the initial state the following energy values hold:

$\sqrt{s_0} = 5.5034\ GeV$ – energy per one nucleon
$\sqrt{s_{NN}} = 11.0067\ GeV$ – energy per nucleon-nucleon pair
$\sqrt{s_{total}} = 2 \cdot A \cdot \sqrt{s_0} = 2 \cdot 197 \cdot 5.5034 = 2168\ GeV$ – energy of two gold nuclei

One of the tasks of this work is the study of the dependence of the energy released during the collision on the impact parameter. The impact parameter (b) in this model is the distance between the centres of the two gold nuclei. Let us consider the formulas used for data processing:

Total energy of the particles produced during the collision [Appendix 1]:

$$\sqrt{s_{eff}}(b) = \frac{N_{part}}{2A}\sqrt{s_{total}} = N_{part} * \sqrt{s_0} \tag{1}$$

Energy of each particle separately

$$energy_{csm} = \sqrt{p_x^2 + p_y^2 + p_z^2 + m^2} \tag{2}$$

Expected dependence $\sqrt{s_{eff}}(b)$:

$$\sqrt{s_{eff}} = C \cdot (R^2 - b^2)^2 \tag{3}$$

where C is a constant, R is the radius of gold, b is the impact parameter [Appendix 1].

3. Implementation

Having considered the structure of the detector system, as well as the physics of the process (propagation of particles from the collision point to the plane of the detector), let us move on to the formation of the feature vector fed to the input of the neural network. The feature vector contains a numerical representation of the collision event, including important parameters that can be extracted from the signal obtained from the microchannel plate detectors. These parameters play a key role in the process of reconstruction of collider events, as well as in the analysis of quantities of interest to researchers. In the context of this work, the main attention is paid to the energy released during the collision.

The feature vector is formed on the basis of three main quantities measured for each particle registered by the detector:

1. Time of flight (tof). Knowing the energy and momentum of a particle, one can calculate its velocity and time of flight and, as a consequence, draw a conclusion about its nature – for example, distinguish light particles (electrons, muons) from heavier ones (protons, pions).

2. Number of the triggered pad (pad number). This parameter shows where exactly on the surface of the detector the particle hit.

3. Detector number (detector ID). In our configuration there are two detectors: to the left and to the right of the collision point. Which detector the particle hit will be determined by the direction of the momentum projection p_z: if $p_z > 0$, then detector ID = 1, if $p_z < 0$, then detector ID = -1.

To reconstruct the coordinates of the collision of a particle with the detector, we assume that the trajectory of the particles can be linearly approximated. Let us introduce the particle motion parameter t, related to the momentum $\vec{p} = (p_x, p_y, p_z)$:

$$\begin{cases} x = t \cdot p_x \\ y = t \cdot p_y \\ z = t \cdot p_z \end{cases},$$

Since the detector is located in the plane $z_0 = |z| = 400$ cm, the motion parameter is given by the following expression: $t = \frac{z_0}{p_z}$. Knowing this parameter, it is possible to determine the coordinates of the point of collision of the particle with the detector:

$$x = \frac{z_0}{p_z} \cdot p_x, \qquad y = \frac{z_0}{p_z} \cdot p_y$$

Obviously, the larger p_z, the faster the particle will reach the plane of the detector. At $p_z = 0$ the particle will never reach the detector.

Using the obtained coordinates x, y it is possible to determine the second parameter of the feature vector – the pad number. The microchannel plate detector is divided into rings and sectors. Each pad corresponds to a unique pair of indices – the ring number and the sector number, which makes it possible to uniquely identify the region of signal registration. To determine the pad number it is necessary to pass from the Cartesian coordinate system (x, y) to the polar coordinate system (r, φ), where r is the distance from the centre of the detector to the point where the particle hits the detector, and φ is the azimuthal angle measured from the Ox axis. Mathematically the transition is given by the expression: $r = \sqrt{x^2 + y^2}$, $\varphi = \arctan\left(\frac{y}{x}\right)$. Before determining the ring and sector number, it is necessary to discretise the space: divide the whole range of radii and angles into uniform intervals (the radius is divided into ring zones, and the angle into sectors), and then see which interval the value of r and φ falls into. As a result we obtain a pair of numbers that uniquely identifies each pad: (i_{ring}, i_{sector}). If the particle did not hit the sensitive surface of the detector, then the parameters of the particle in the feature vector are filled with zeros.

As was already noted earlier, the most important physical parameter registered in the experiment is the time of flight – tof, which is calculated as:

$$tof = \frac{z_0}{v}, \quad \text{where} \quad v = \frac{p}{energy_{csm}} \cdot c$$

Here $p = \sqrt{p_x^2 + p_y^2 + p_z^2}$ is the total momentum of the particle, $energy_{csm}$ is the energy of each particle separately, and $c = 29.98\ \frac{\text{cm}}{\text{ns}}$ is the speed of light. For a more convenient physical interpretation of the process, the following parameter is introduced:

$$T = \frac{z_0}{tof - t_0},$$

where $t_0 = \frac{z_0}{c}$ is the time in which light reaches the surface of the detector. That is, particles flying almost at the speed of light (light and fast) will have very large values of T. And, conversely, particles with low velocities and, as a consequence, a large value of tof will have a small value of T. This is an important characteristic of the process, which can help in the further analysis of the event by determining the type of particle.

Thus, a feature vector of size $[3, 120]$ will be fed to the input of the neural network, where 120 is the maximum number of particles that can be registered by the two detectors simultaneously ($2 \cdot rings \cdot sectors = 120$), and 3 are the features characterising each particle separately (T, pad number, detector ID).

The next stage of the work, after determining the structure of the feature vector, was the preparation of the model data for the further analysis of the dependence $\sqrt{s_{eff}}(b)$ and the construction of a function capable of mapping the signal from the detector to the energy released after the collision. The processing of the data (as well as all further stages of the work) obtained from the QGSM event generator, which models collisions of heavy ions, was implemented in the Python programming language. Initially the work was carried out with 10 files, each containing 2000 events. In the course of the work, because of overfitting of the model, it was decided to increase the amount of data to 100 files (200 000 events in total). This improved the quality of training of the neural network on the training datasets, and also helped the model to achieve more accurate prediction results on the test samples.

In the course of the work a number of specialised libraries were also used:

- NumPy [6] – was used for working with multidimensional arrays, as well as for efficient and fast mathematical computations
- Pandas [7] – was applied for reading and structuring files in csv and txt formats
- Matplotlib [8] – provided visualisation of the obtained results
- PyTorch [9] – was the main platform for building, training and evaluating the neural network model
- Scipy [10] – was used for working with distributions and constructing approximation functions in the analysis of the statistical properties of errors
- Scikit-learn [11] – was applied for splitting the data into training and test samples, as well as for computing metrics

	1	10	14.400	2.781	14.129					
1	0	1	0	321	7.4161E-01	7.6406E-01	-4.2361E-01	4.7175E+00	4.2361E-01	4.93670E-01
0	0	0	0	111	-2.2660E-01	5.0864E-02	2.1669E-01	3.2592E+00	-2.1669E-01	1.34960E-01
0	0	-1	0	130	2.6345E-01	-3.8472E-01	4.0930E-01	6.9840E+00	-4.0930E-01	4.97670E-01
-1	0	0	0	-211	4.9467E-02	6.9877E-02	8.8307E-02	1.5900E+00	-8.8307E-02	1.39570E-01
1	0	0	1	2212	4.0255E-02	-4.6151E-01	3.4456E+00	4.0942E+01	-3.4456E+00	9.38280E-01
1	0	0	1	2212	-5.8720E-01	2.6631E-01	-4.2285E+00	5.0475E-01	4.2285E+00	9.38280E-01
1	0	0	1	2212	-2.3434E-01	-3.3428E-01	6.0368E+00	7.0660E+01	-6.0368E+00	9.38280E-01
0	0	0	1	2112	-2.8048E-02	2.3955E-02	5.3521E+00	6.2678E+01	-5.3521E+00	9.39570E-01
77	0	0	194	1000771940	-1.6648E-02	7.9281E-02	1.0519E+03	1.2315E+04	-1.0519E+03	1.80677E+02
78	0	0	196	1000781960	-5.8202E-03	-5.4375E-02	-1.0627E+03	-1.6641E+00	1.0627E+03	1.82540E+02
	2	1153	6.668	5.918	-3.073					
0	1	0	0	22	-2.0510E-01	-3.7953E-02	-2.0267E-01	4.9110E-01	2.0267E-01	0.00000E+00
0	1	0	0	22	-4.0834E-01	9.1391E-02	3.7699E-01	5.4558E+00	-3.7699E-01	0.00000E+00
0	1	0	0	22	-6.0982E-02	1.8251E-01	-1.4767E-01	5.3465E-01	1.4767E-01	0.00000E+00
0	1	0	0	22	2.3742E-01	5.0139E-02	-5.6060E-01	2.4171E-01	5.6060E-01	0.00000E+00
0	1	0	0	22	-1.6874E-01	3.3044E-01	-2.7939E-01	1.0435E+00	2.7939E-01	0.00000E+00
0	1	0	0	22	5.0975E-01	-2.2054E-01	5.5168E-01	7.7452E+00	-5.5168E-01	0.00000E+00
0	1	0	0	22	-7.6361E-02	-8.9971E-02	5.2503E-01	6.1777E+00	-5.2503E-01	0.00000E+00
0	1	0	0	22	3.0057E-01	3.9997E-02	1.1649E+00	1.3763E+01	-1.1649E+00	0.00000E+00
0	1	0	0	22	-9.3515E-02	-1.6935E-01	1.5998E-01	2.3846E+00	-1.5998E-01	0.00000E+00
-1	1	0	0	-211	-3.5216E-02	9.6279E-02	9.0849E-02	1.3220E+00	-9.0849E-02	5.11003E-04
0	1	0	0	22	-1.4942E-01	-1.3503E-01	8.2596E-01	9.7393E+00	-8.2596E-01	0.00000E+00
1	1	0	0	211	-1.8900E-01	2.6198E-01	5.3703E-01	6.7588E+00	-5.3703E-01	5.11003E-04
0	1	0	0	22	1.3617E-01	-5.5119E-02	2.7538E-02	1.0234E+00	-2.7538E-02	0.00000E+00
0	1	0	0	22	1.6661E-01	-2.9226E-01	1.5088E+00	1.7749E+01	-1.5088E+00	0.00000E+00
0	1	0	0	22	1.8406E-01	-1.6032E-01	6.0495E-01	7.3044E+00	-6.0495E-01	0.00000E+00
0	1	0	0	22	-3.9315E-01	-1.6070E-01	1.1448E+00	1.3746E+01	-1.1448E+00	0.00000E+00
0	0	1	0	310	-2.0718E-01	6.1771E-03	1.8449E-01	4.3668E+00	-1.8449E-01	4.97670E-01
0	0	0	0	111	-8.0204E-02	2.9563E-02	-2.3577E-01	2.6251E-01	2.3577E-01	1.34960E-01
0	0	0	0	111	6.7795E-02	-8.1357E-03	-3.4650E-01	1.5230E-01	3.4650E-01	1.34960E-01
0	0	0	0	111	3.8335E-02	-3.2636E-02	-5.1528E-02	5.8079E-01	5.1528E-02	1.34960E-01

Fig. 2: Initial form of the model data of the QGSM generator

Each file is a structured array in which the data are recorded event by event. Each collision takes place at a certain impact parameter. At the beginning of each event the event number, the number of particles, the impact parameter and its projections on the Ox and Oy axes are given: $b = \sqrt{b_x^2 + b_y^2}$. Then the characteristics of the particles follow line by line. In the course of the work the following quantities were actively used: the charge of the particle (first column), the pdg code of the particle (fifth column), the momentum projections in the centre-of-mass system p_x, p_y, p_z (sixth, seventh, eighth columns), m the mass of the particle (eleventh column). One of the most important stages of data processing is the stage of separating spectators from the other particles. A spectator is a particle that was part of the nucleus even before the collision and practically did not change the trajectory of its motion, as well as its energy. These particles do not carry information about the interaction process itself, and their exclusion makes it possible to estimate the energy of the particles that took part in the interaction. We will call this quantity the released energy.

Two filters were implemented for selecting spectators from the other particles:

1. Energy filter:

$$energy_{csm} \approx \sqrt{s_0} \text{ – condition of equality of energies before and after the collision}$$

2. Momentum filter:

$\left(p_x^2 + p_y^2\right) \ll p_z^2$ – condition of smallness of the transverse component of momentum compared to the longitudinal one

Also, in the course of the work two approaches to preparing the data for the training and test samples were implemented:

1. Use of all particles, regardless of their charge. In this case the widest possible range of events is covered, and the energy released during the collision is considered for all particles produced during the collision

2. Selection of only charged particles. This approach is justified by the fact that real experimental setups (for example detectors on microchannel plates) are capable of registering mainly only charged particles. Thus we can approximately reproduce real experimental conditions.

Comparison of the model's performance in these two cases will help to assess the degree of influence of neutral particles on the final result (the accuracy of predicting the energy released during the collision) and, as a consequence, to determine which dataset is better suited for forming the feature vector.

Let us now proceed directly to the formulation of the regression problem and the description of the model training process.

In this work we aim to train the model in such a way that it can predict with high accuracy a scalar quantity – the energy released as a result of a nuclear collision. That is, we set ourselves the task of teaching the neural network to produce a numerical estimate of the quantity under study. Our model must receive a feature vector at the input, and output only one number – the energy released in the event. The model is trained on labelled data, where each input feature vector is matched with the value of the energy released in the collision.

One training cycle of a neural network model is called an epoch – in one epoch the whole set of training data must pass through the algorithm. To increase the robustness of the process to overfitting and noise, the training sample is first divided into batches (samples of fixed and small size), formed randomly. Each batch is fed to the input of the neural network and processed step by step: the model, on the basis of the input feature vector, forms a prediction of the collision energy, and then the value of the error function is calculated – in this work it is MSE. After calculating the error, the backpropagation mechanism is launched, during which the gradients of the neural network weights are found. These gradients are used for the further correction of the network weights by means of the *Adam* optimiser, which automatically selects the optimal learning rate. After the parameters are updated, the gradients are zeroed, and the procedure is repeated again for the remaining batches from the training sample.

The process described above is repeated until the last epoch is completed. At the end the model is tested on the test sample, consisting of data previously unseen by the model. During this process the ability of the model to generalise the dependencies under study, as well as its ability to make accurate predictions, is evaluated.

In the course of this work two neural network models were considered: a fully connected NN and a neural network with one convolutional layer.

1. Fully connected neural network:

Each linear transformation of the model has the following form:

$$\vec{t}^{(0)} = \vec{x}, \qquad \vec{t}^{(i+1)} = \sigma\left(W_i\vec{t}^{(i)} + \vec{b}^{(i)}\right), \qquad i = 0,\ldots,N$$

- $\vec{x} \in \mathbb{R}^{360}$ – input feature vector,
- $\vec{t}^{(i)} \in \mathbb{R}^{N_i}$ – output of the i-th layer,
- $W_i \in \mathbb{R}^{N_{i+1}\times N_i}$ – weight matrix of the layer,
- $\vec{b}^{(i)} \in \mathbb{R}^{N_{i+1}}$ – bias vector,
- $\sigma(\cdot)$ – SiLU activation function, where $\sigma(x) = x \cdot \frac{1}{1+e^{-x}}$,

In this architecture $N = 3$ hidden layers are used. The output layer looks as follows:

$$\hat{y} = W_{N+1}\vec{t}^{(N+1)} + \vec{b}^{(N+1)}$$

Let us also consider the table of hyperparameters, valid both for this neural network architecture and for the next one:

Table 1: Hyperparameters of the fully connected neural network

Activation function	Loss function	Optimizator	Weight decay	Learning rate	Batch size	Epochs
SiLU	MSE	Adam	10^{-5}	0.001	3000	1500

2. Network with a convolutional layer:

The convolutional layer applies the convolution operation to the input tensor $X \in \mathbb{R}^{1\times H_{in}\times W_{in}}$, where 1 is the input channel, H_{in} is the height, and W_{in} is the width. In the work under consideration $H_{in} = 120$, and $W_{in} = 3$ – for each particle we consider the feature vector (tof, pad number, detector ID). In the work the input tensor X is processed by a two-dimensional convolution with 8 filters and a kernel of dimension 6×3 (with the filter we pass over 6 particles at a time, and for each particle we take the feature vector). This transformation can be written in the following form:

$$F_k = ReLU\left(\sum_{c=1}^{I} X_c * K_{k,c} + b_k\right), \qquad k = 1,\ldots,K$$

I is the number of input channels ($I = 1$), K is the number of filters ($K = 8$), $K_{k,c} \in \mathbb{R}^{6\times 3}$ are the convolution kernels, $X_c \in \mathbb{R}^{120\times 3}$ is the c-th input channel, $*$ is the operation of two-dimensional convolution, $F_k \in \mathbb{R}^{H_{out}\times W_{out}}$ is the result of applying the k-th filter, ReLU is the activation function. Let us calculate the values H_{out} and W_{out} using the following formula:

$$output_{size} = input_{size} - kernel_{size} + 1.$$

Then at the output we obtain that:

$$H_{out} = 120 - 6 + 1 = 115, \quad W_{out} = 3 - 3 + 1 = 1 \;\Rightarrow\; F_k \in \mathbb{R}^{115\times 1}$$

After applying all 8 filters we obtain the tensor $F \in \mathbb{R}^{K\times H_{out}\times W_{out}}$ ($\mathbb{R}^{8\times 115\times 1}$), which is then flattened by means of the flatten function into the vector $\vec{f} \in \mathbb{R}^{1840}$. The flattening operation is necessary in order to transform the tensor obtained after the convolution into a form that can be fed to the input of the neural network.

Next, the vector $\vec{f}$ passes successively through a series of fully connected layers, already considered for the first model.

4. Verification of the Fulfilment of the ECL and MCL

From the theoretical background we know the value of the energy before the collision:

$$E_0 = \sqrt{s_{total}} = 2168\ GeV$$

Consequently, to check the fulfilment of the ECL it is necessary to calculate the energy after the collision – the sum of the energies of the spectators and the produced particles. We have the model data taken from the QGSM generator (pdg is the particle code, the mass of the particle, the momentum components in the centre-of-mass system). Let us use formula (2) to calculate $E_1 = \sum energy_{csm}$.

Let us introduce the quantity:

$$\eta = \frac{E_1 - E_0}{E_0},$$

where η is the relative deviation of the quantity E_1 from E_0. The value E_0 is the expected mean value of the quantity E_1 (averaging over different impact parameters), therefore the fulfilment of the ECL is equivalent to the smallness of the mean value of η.

Let us construct a histogram, plotting along the Ox axis the value of η, and along the Oy axis the number of hits in the given interval among all possible values of the impact parameter (we have a finite number of them – 2000 points).

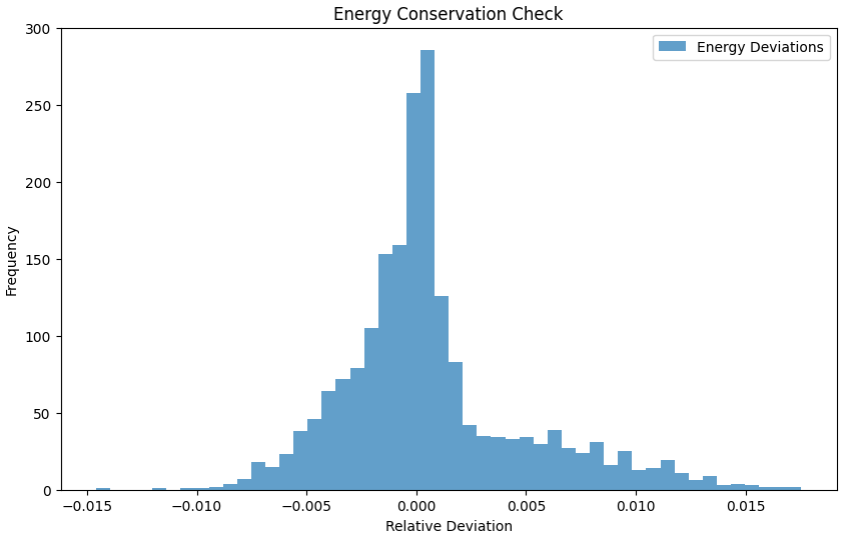


Fig. 1: Distribution of events depending on η

As can be seen from the histogram in Fig. 1, the main part of the events is concentrated in the immediate vicinity of zero, which indicates the fulfilment of the law of conservation of energy with high accuracy. The distribution with respect to η has a clearly pronounced peak near the zero value, as well as symmetrically decreasing "tails". Let us also note that almost all points lie in the interval [-0.015; 0.015]. That is, the CL for energy is fulfilled within 1.5%.

Now let us check the fulfilment of the law of conservation of momentum for each component of momentum separately. To check the MCL we need the following condition to be fulfilled: $\vec{p}_{initial} = \Sigma_i \vec{p}^{\,i}_j$, where $j = x, y, z$, and i is a specific event ($i \in [1; 2000]$), and $\vec{p}_{initial} = 0$. This is the value of the momentum at the initial moment of time (before the collision) in the centre-of-mass system. Let us construct the plots for p_x, p_y, p_z:

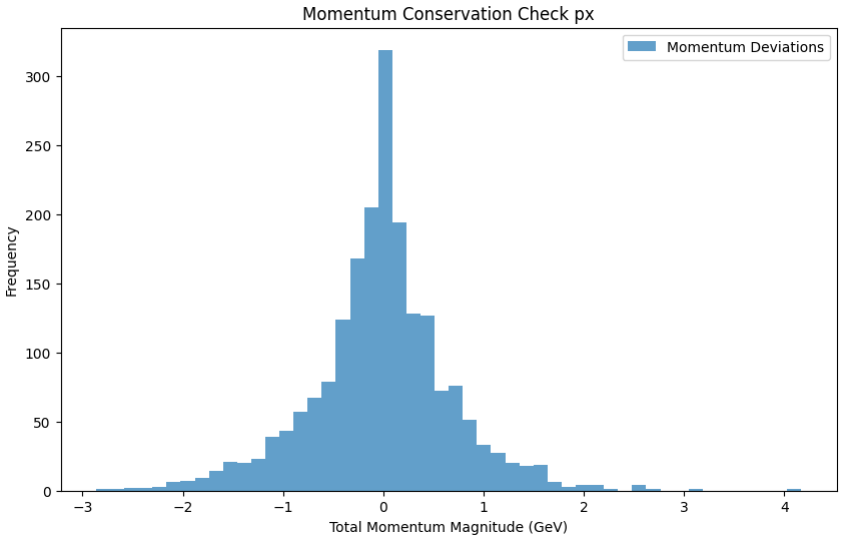


Fig. 2: Distribution of particles over the momentum component p_x

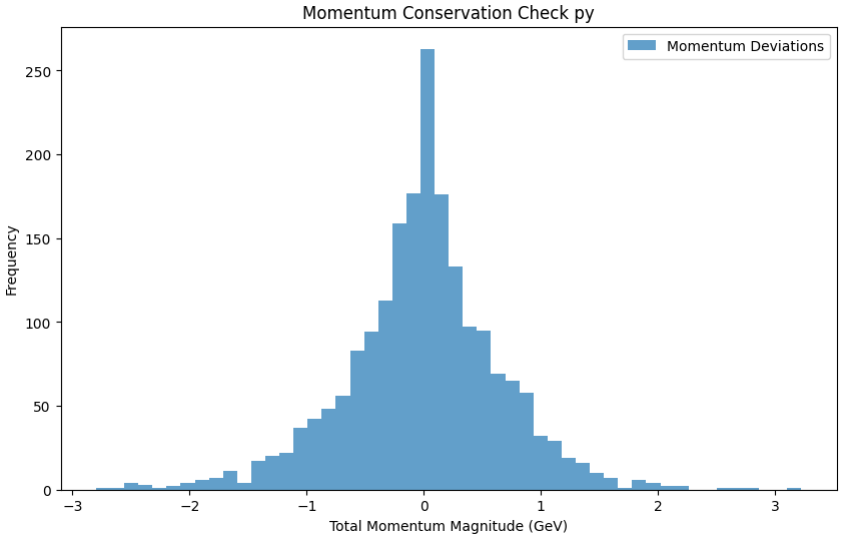


Fig. 3: Distribution of particles over the momentum component p_y

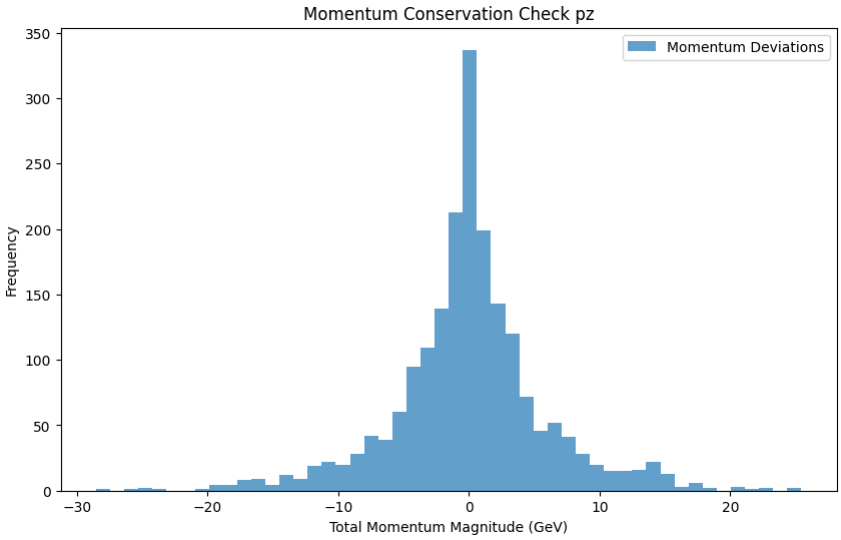


Fig. 4: Distribution of particles over the momentum component p_z

From the plots it can be seen that the MCL is fulfilled better for the components p_x and p_y.

5. Energy and Momentum Filters

By the definition used, particles produced after the collision cannot be spectators. Based on this, let us use sorting by particles, that is, we will take only nucleons and heavy nuclei. We will determine spectators in two ways:

- The energy before and after the collision is the same
- The transverse component of momentum is much smaller than the longitudinal one

Both of the mentioned methods require specifying an error (under ideal conditions it tends to 0%).

For the energy method of determining spectators, we will call the error the value of the deviation from the theoretical value:

$$RelErr = \frac{E_0 - \Delta E}{E_0}$$

Thus, we consider a particle to be a spectator if it is a nucleon or a heavy nucleus, and also its energy falls into the following interval: $[E_0 - \Delta E; E_0 + \Delta E]$

For the method of determining spectators through the momentum components, we will call the error the quantity:

$$RelErr1 = \frac{\sqrt{p_x^2 + p_y^2}}{|p_z|}$$

Thus, we consider a particle to be a spectator if it is a nucleon or a heavy nucleus, and also its transverse component of momentum falls into the following interval: $[0; RelErr1 \cdot p_z]$

6. Construction and Analysis of the Dependence of the Interaction Energy on the Impact Parameter

First, let us try different values of the errors (choosing $RelErr = RelErr1$) for both filters. In the plots, the blue colour shows the energy of the particles produced during the collision as a function of the impact parameter, and the green colour shows the energy of the spectators as a function of the impact parameter.

Error 6%:

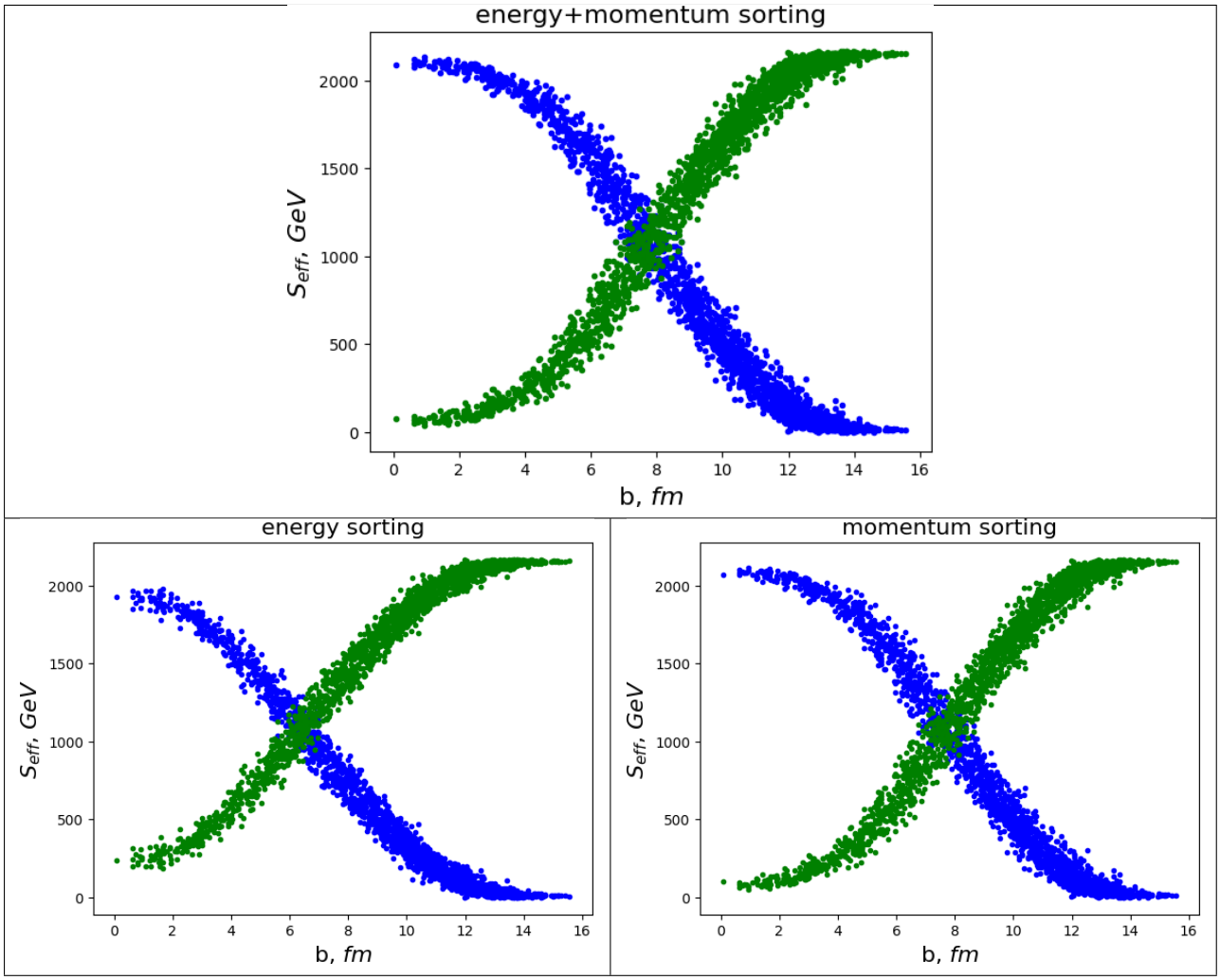


Fig. 5: Dependence $\sqrt{s_{eff}}(b)$ at $Err = 6\%$. The blue colour shows the energy of the particles produced during the collision as a function of the impact parameter, the green colour shows the energy of the spectators as a function of the impact parameter.

Error 10%:

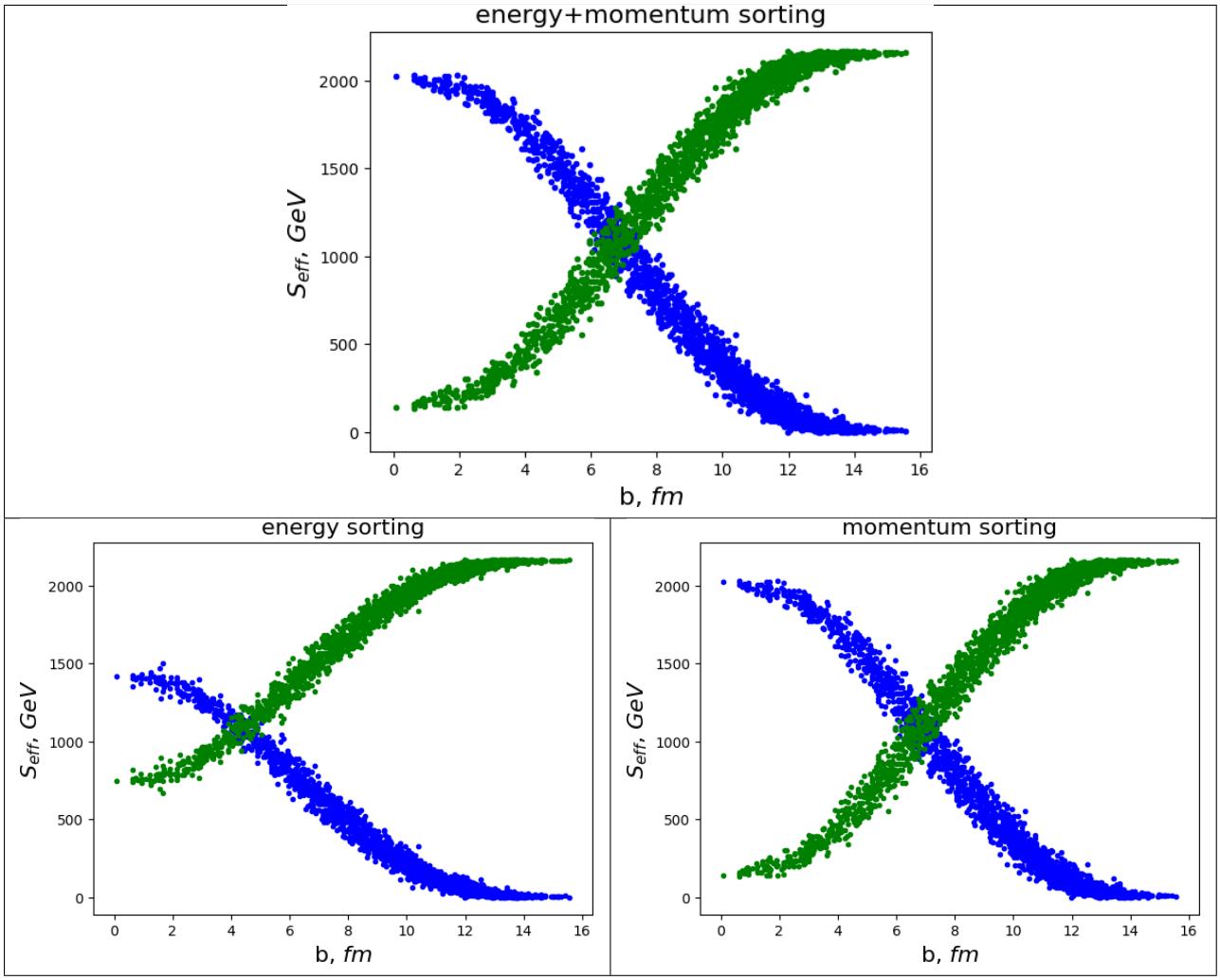


Fig. 6: Dependence $\sqrt{s_{eff}}(b)$ at $Err = 10\%$. The blue colour shows the energy of the particles produced during the collision as a function of the impact parameter, the green colour shows the energy of the spectators as a function of the impact parameter.

At an error of 6% and more it can be seen that the plots obtained when using the filter for selecting spectators by energy change strongly: in a head-on collision ($b = 0$), the energies of the spectators are not equal to zero, and this contradicts the theory.

Let us now try to take a very small error: $Err = 0.1\%$

Error 0.1%:

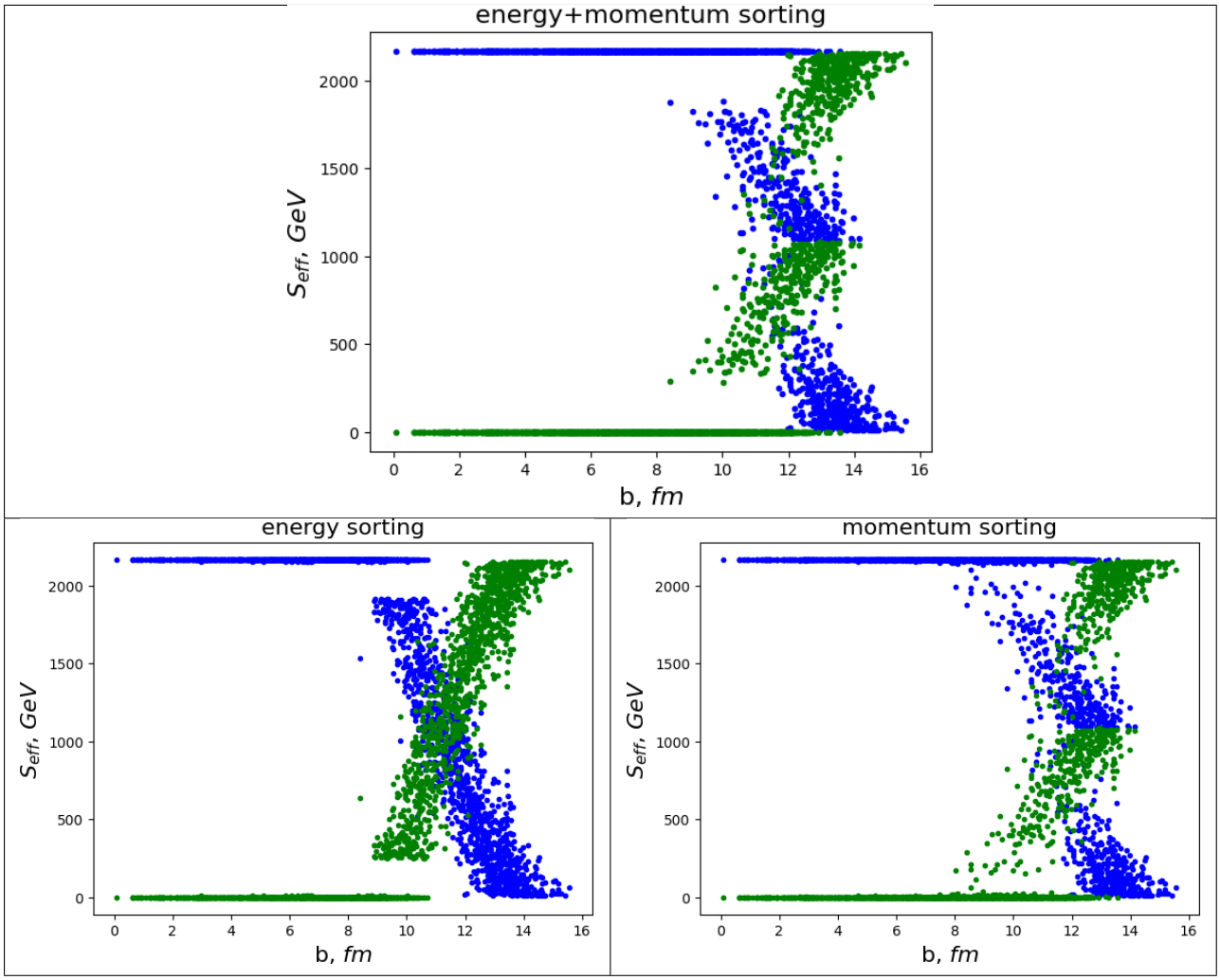


Fig. 7: Dependence $\sqrt{s_{eff}}(b)$ at $Err = 0.1\%$. The blue colour shows the energy of the particles produced during the collision as a function of the impact parameter, the green colour shows the energy of the spectators as a function of the impact parameter.

As can be seen from the plot, the obtained dependences are not interpreted physically and do not reflect the expected dependence $\sqrt{s_{eff}}(b)$. Such behaviour may be caused by the features of the algorithms used in the QGSM generator.

From the conservation laws we noticed that the main part of the points for energies lies in the interval 1.5%, and for momentum in the interval 14%. When taking into account an error of 14% we obtain plots that are poorly interpreted physically. This may be related to the inaccuracy of the model data. By going through various errors for the momentum filter, it was found that at an error of 2% the best plots are obtained. Let us construct, taking into account such errors, the plots of the dependence of energies on the impact parameter.

Error 1.5% in energy and 2% in momentum:

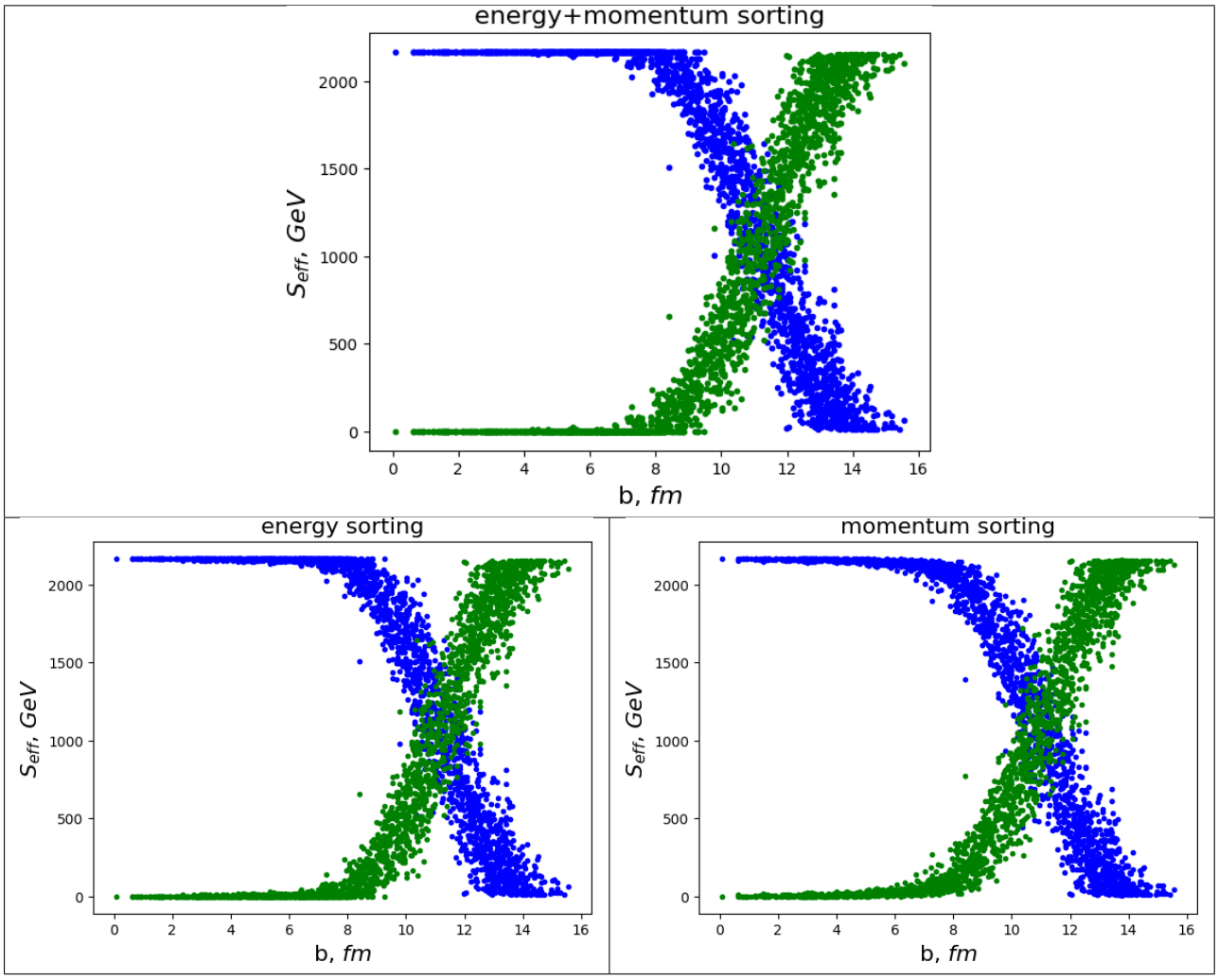


Fig. 8: Dependence $\sqrt{s_{eff}}(b)$ at $Err = 1.5\%$ and 2%. The blue colour shows the energy of the particles produced during the collision as a function of the impact parameter, the green colour shows the energy of the spectators as a function of the impact parameter.

As can be seen, the plots coincide in their form with the expected ones: the energy of the spectators grows rapidly at an impact parameter of the order of the size of the nucleus.

7. Construction of a Function Mapping the Signal from the Detector to the Released Energy

At the final stage of the work, two neural network models were built and trained (a fully connected NN, as well as an NN with the addition of one convolutional layer), whose task is the reconstruction of the energy released during the collision of two gold nuclei on the basis of the signal obtained from the microchannel plate detectors. Below are the results of both models for two different cases: when the detector is capable of registering only charged particles and when the detector registers absolutely all particles that reached it.

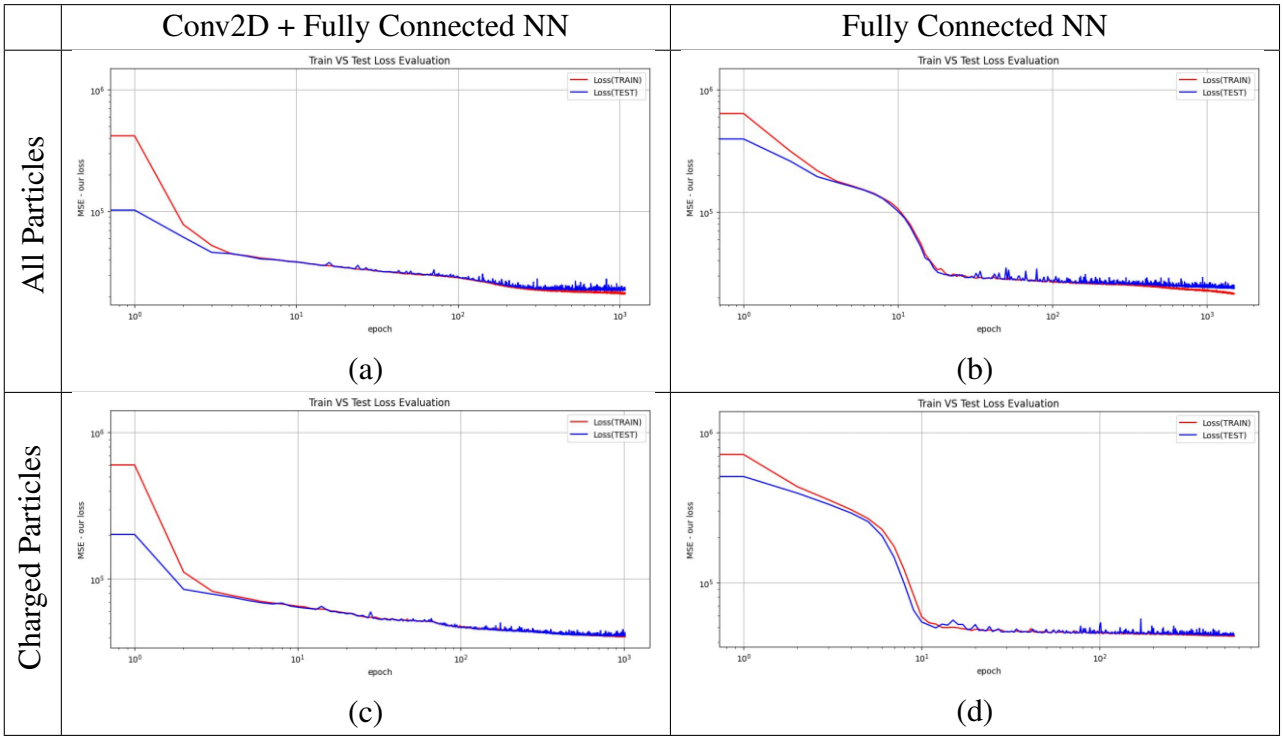


Fig. 9: Evolution of the loss function over time on the test and training samples: a – all particles and hybrid NN, b – all particles and fully connected NN, c – charged particles and hybrid NN, d – charged particles and fully connected NN.

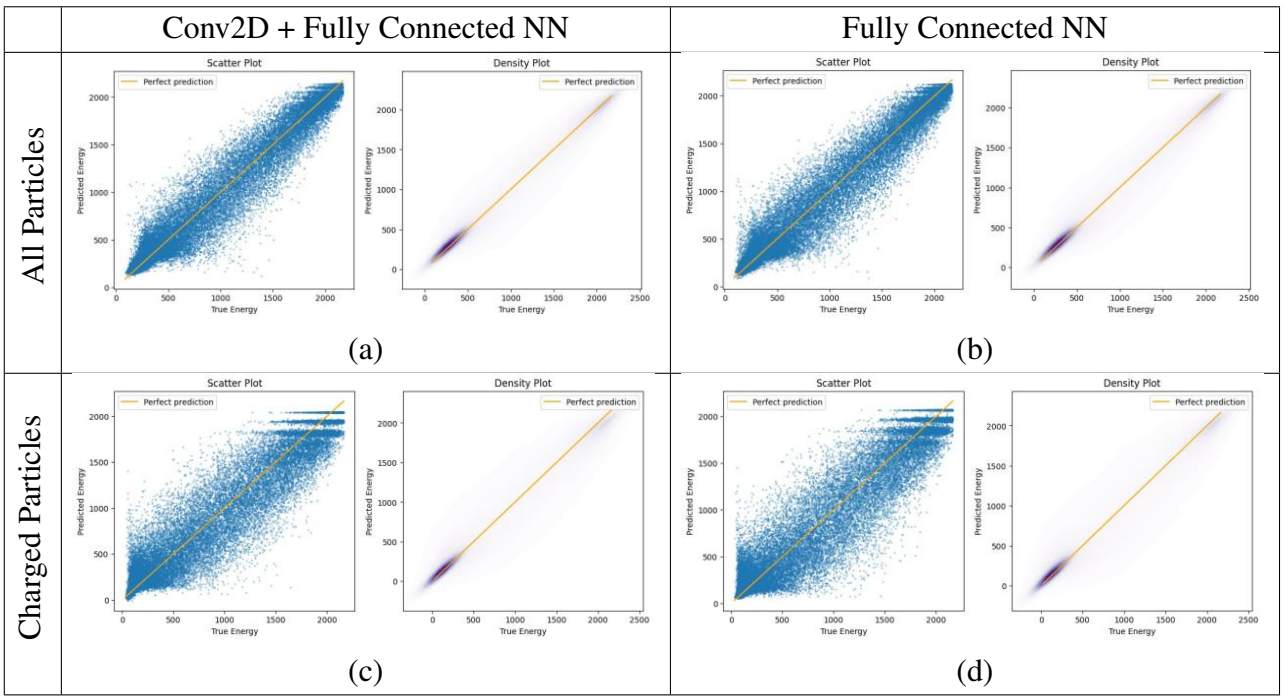


Fig. 10: Dependence of the predicted energy on the true one, as well as the plot of the density of the energy distribution: a – all particles and hybrid NN, b – all particles and fully connected NN, c – charged particles and hybrid NN, d – charged particles and fully connected NN.

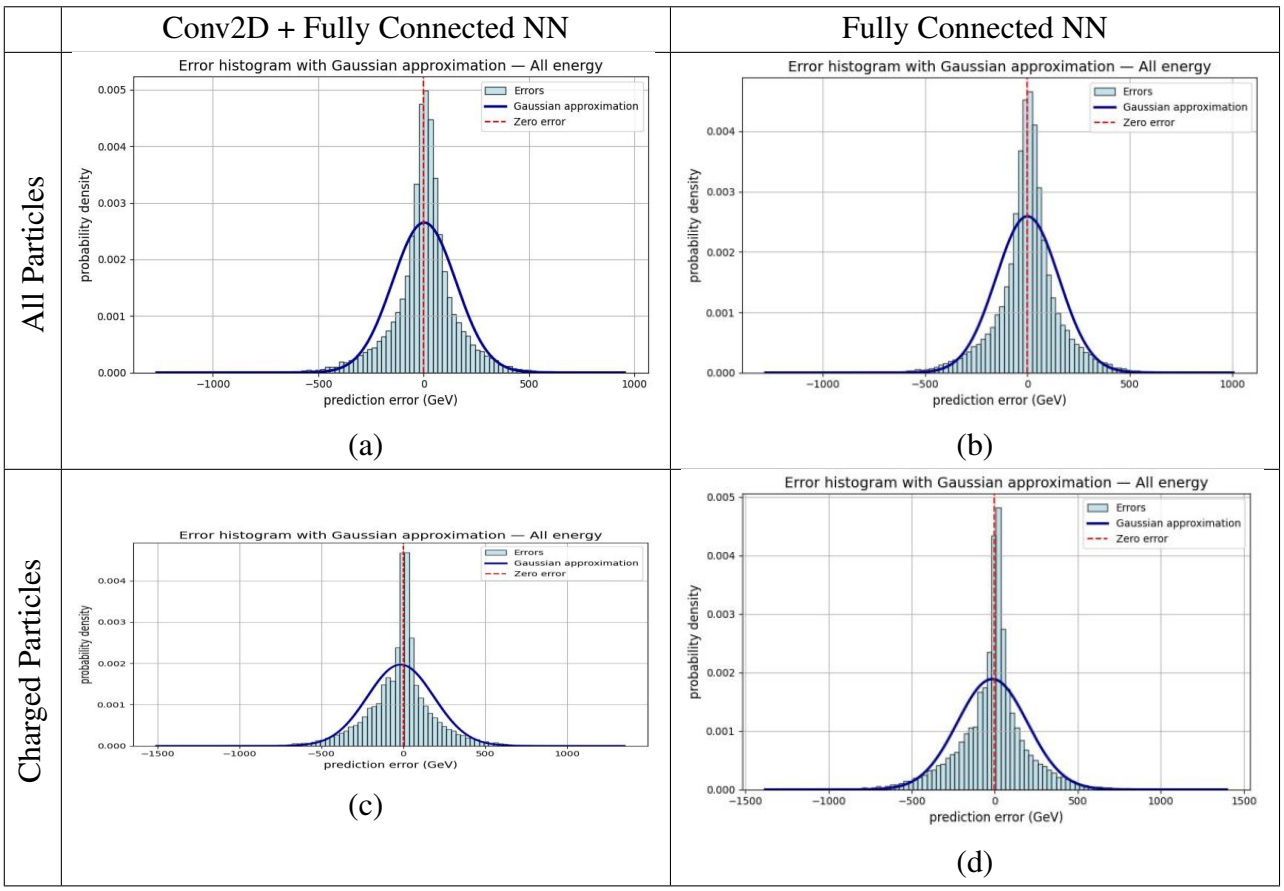


Fig. 11: Histogram of the distribution of the prediction error with Gaussian approximation: a – all particles and hybrid NN, b – all particles and fully connected NN, c – charged particles and hybrid NN, d – charged particles and fully connected NN.

Fig. 9 shows the plots of the change of the error function during training on the training and test samples. The plots show how the model behaves with an increasing number of epochs, and make it possible to judge the convergence and the generalisation ability of the neural network. For a quantitative evaluation of the quality of energy reconstruction, the root mean squared error RMSE was used, given by the formula:

$$RMSE = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2},$$

where:

- $\hat{y}_i$ is the value of energy predicted by the model,
- y_i is the true value of energy,
- N is the number of events in the test sample.

On the basis of this metric it is possible to compare the results of the two models for two different cases. Below are the results of the RMSE calculation:

Table 2: RMSE values in GeV

	Conv2D + Fully Connected NN	Fully Connected
All Particles	150.5	154.0
Charged Particles	203.3	211.6

As can be seen from Table 2, when the model is trained on all particles, including neutral ones, the neural network shows higher accuracy. Although in real conditions microchannel plate detectors register only charged particles, taking into account all particles during training is a theoretical model that allows the neural network to better understand the relation of the energy released in the event with the feature vector.

To evaluate the quality of predictions, in addition to the absolute value of RMSE, let us calculate the relative error using the formula:

$$\varepsilon = \frac{RMSE}{\sqrt{s_{total}}} \cdot 100\%$$

Table 3: Relative error ε

	Conv2D + Fully Connected NN	Fully Connected
All Particles	6.95%	7.10%
Charged Particles	9.38%	9.77%

The obtained values of the relative error show that the minimum error is achieved when all particles are used for training the neural network in the model with the addition of a convolutional layer – $\varepsilon = 6.95\%$. This is the best result among all configurations. The maximum error $\varepsilon = 9.77\%$ is observed for the fully connected neural network trained only on charged particles.

Thus, it can be concluded that using all particles for training gives a better result, which confirms the importance of taking neutral particles into account when analysing the collision energy. Also, the architecture using a convolutional layer in both cases (all particles and only charged ones) consistently gives a smaller error than the fully connected NN. At the same time, the values of ε in all cases are within an acceptable range of values and do not exceed 10%, which indicates that the model predicts the quantity of interest to us quite well, even on the basis of data collected from a very small solid angle.

As the histograms of the error distribution on the full test sample showed (the whole energy range, Fig. 11), the obtained distribution is not quite well approximated by the normal distribution. This is especially noticeable in the thickening of the "tails", as well as in the region of zero error. Such features may be related to the fact that the model copes with predicting the quantity with different accuracy in different numerical ranges of energies.

Let us divide the energy range into three equal parts in order to examine more deeply the following intervals: low-energy events (Low Energy), the region of medium energies (Mid Energy), high-energy energies (High Energy):

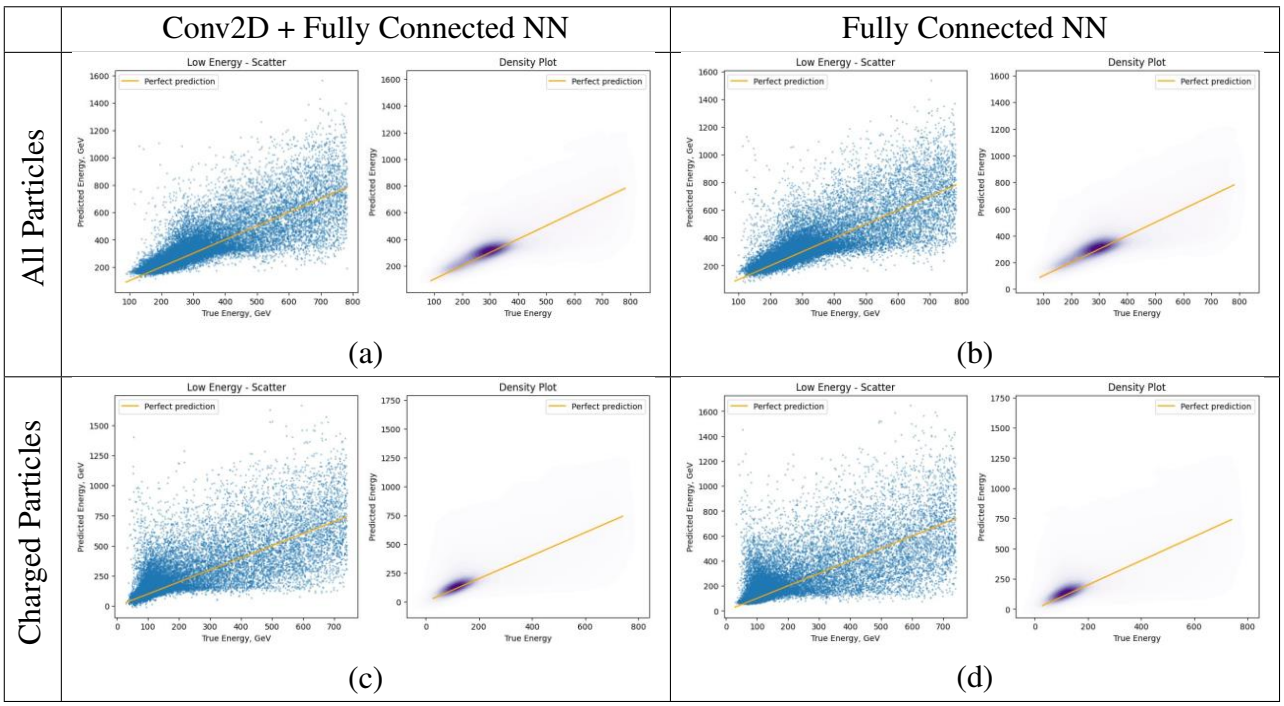


Fig. 12: Dependence of the predicted energy on the true one, as well as the plot of the density of the energy distribution (Low-energy collisions): a – all particles and hybrid NN, b – all particles and fully connected NN, c – charged particles and hybrid NN, d – charged particles and fully connected NN.

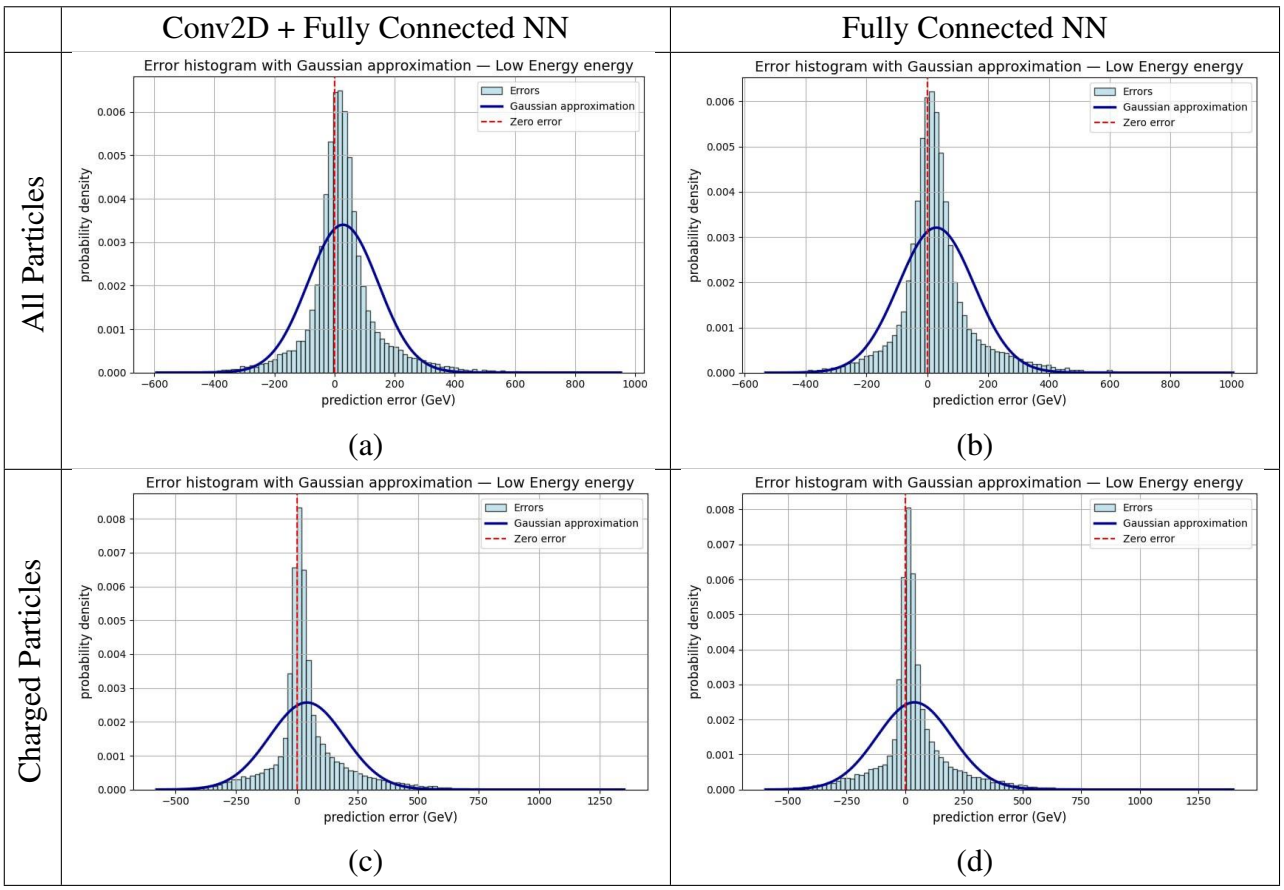


Fig. 13: Histogram of the distribution of the prediction error with Gaussian approximation (Low-energy collisions): a – all particles and hybrid NN, b – all particles and fully connected NN, c – charged particles and hybrid NN, d – charged particles and fully connected NN.

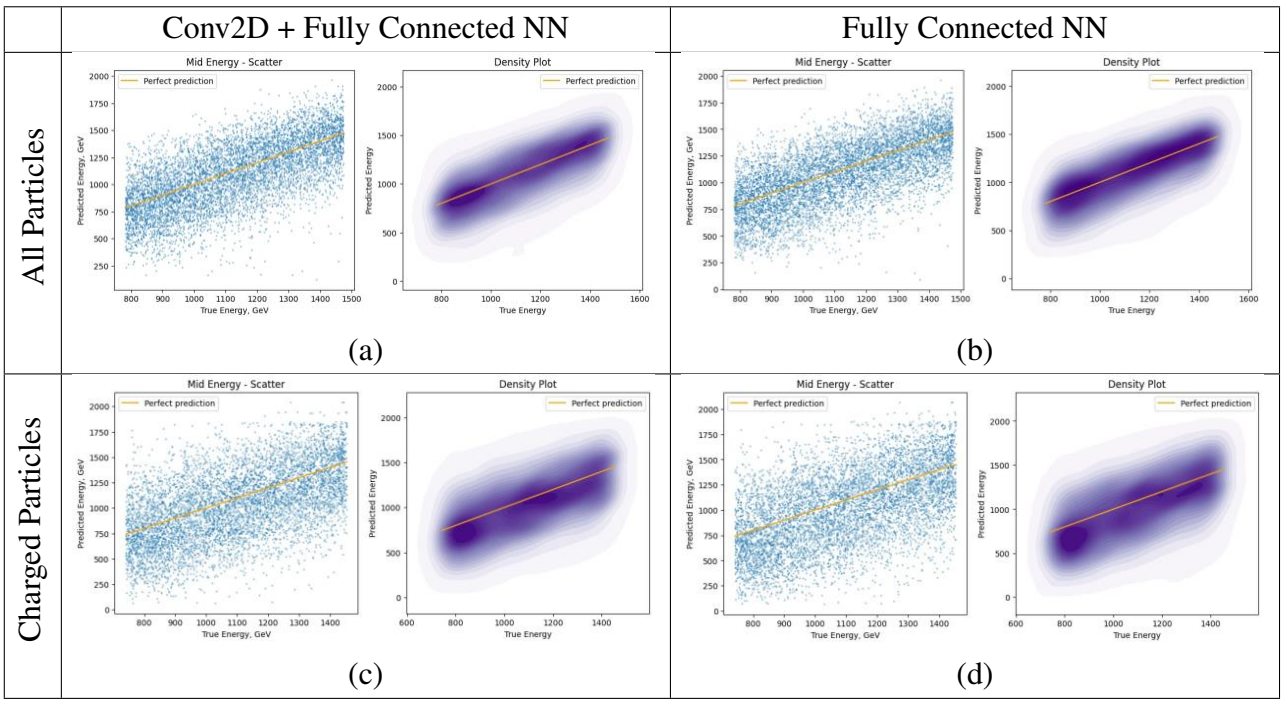


Fig. 14: Dependence of the predicted energy on the true one, as well as the plot of the density of the energy distribution (Collisions with medium energy): a – all particles and hybrid NN, b – all particles and fully connected NN, c – charged particles and hybrid NN, d – charged particles and fully connected NN.

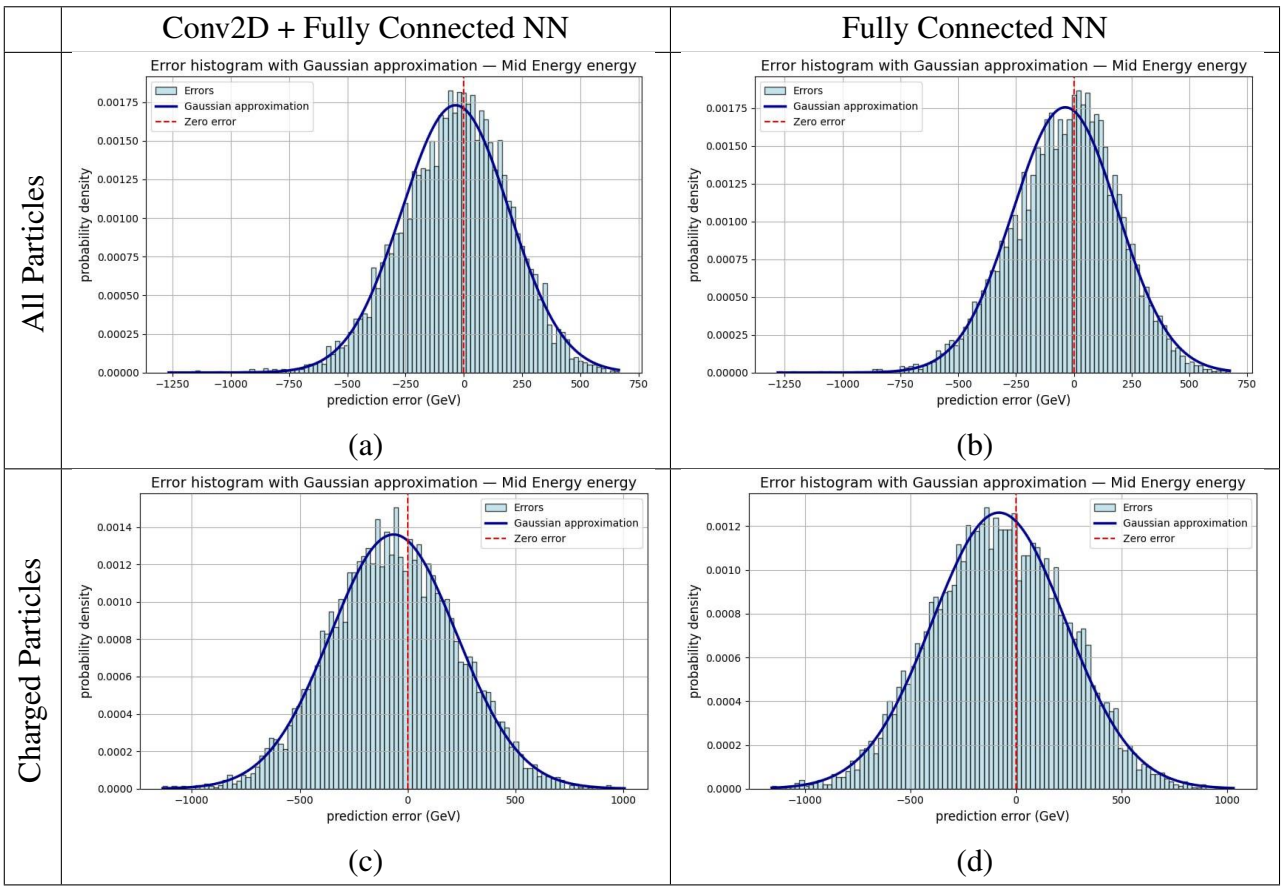


Fig. 15: Histogram of the distribution of the prediction error with Gaussian approximation (Collisions with medium energy): a – all particles and hybrid NN, b – all particles and fully connected NN, c – charged particles and hybrid NN, d – charged particles and fully connected NN.

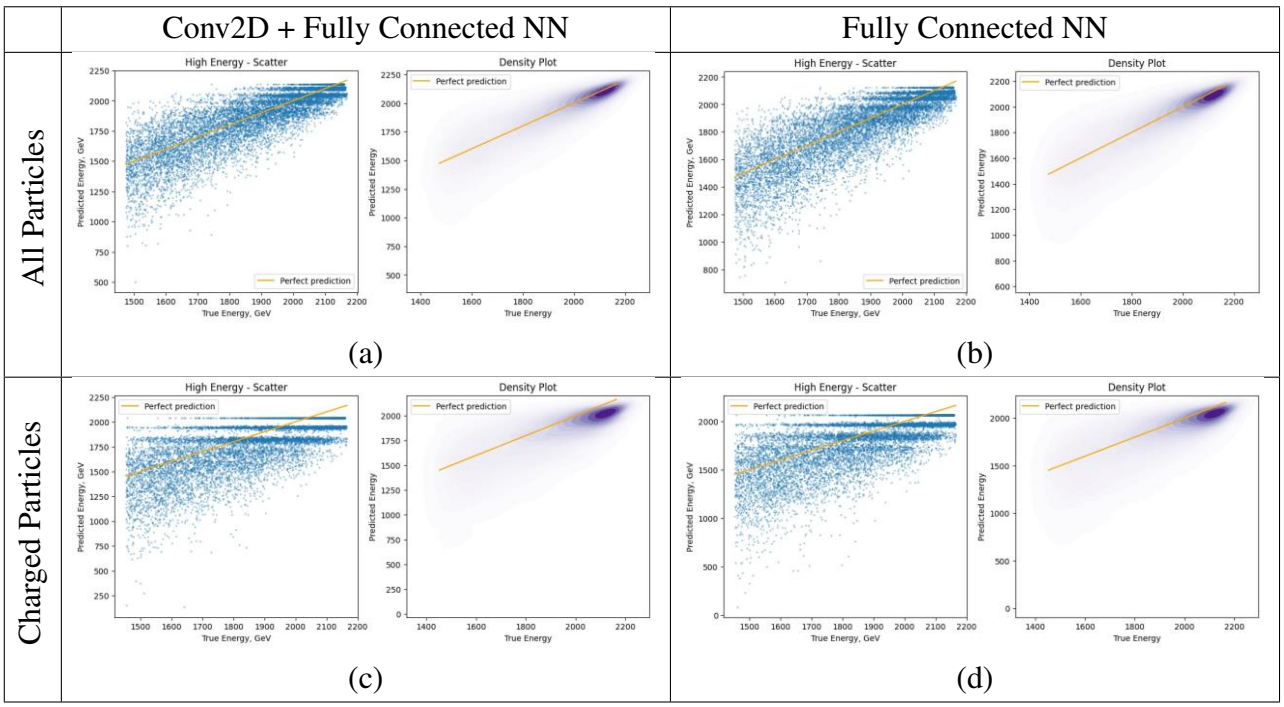


Fig. 16: Dependence of the predicted energy on the true one, as well as the plot of the density of the energy distribution (High-energy collisions): a – all particles and hybrid NN, b – all particles and fully connected NN, c – charged particles and hybrid NN, d – charged particles and fully connected NN.

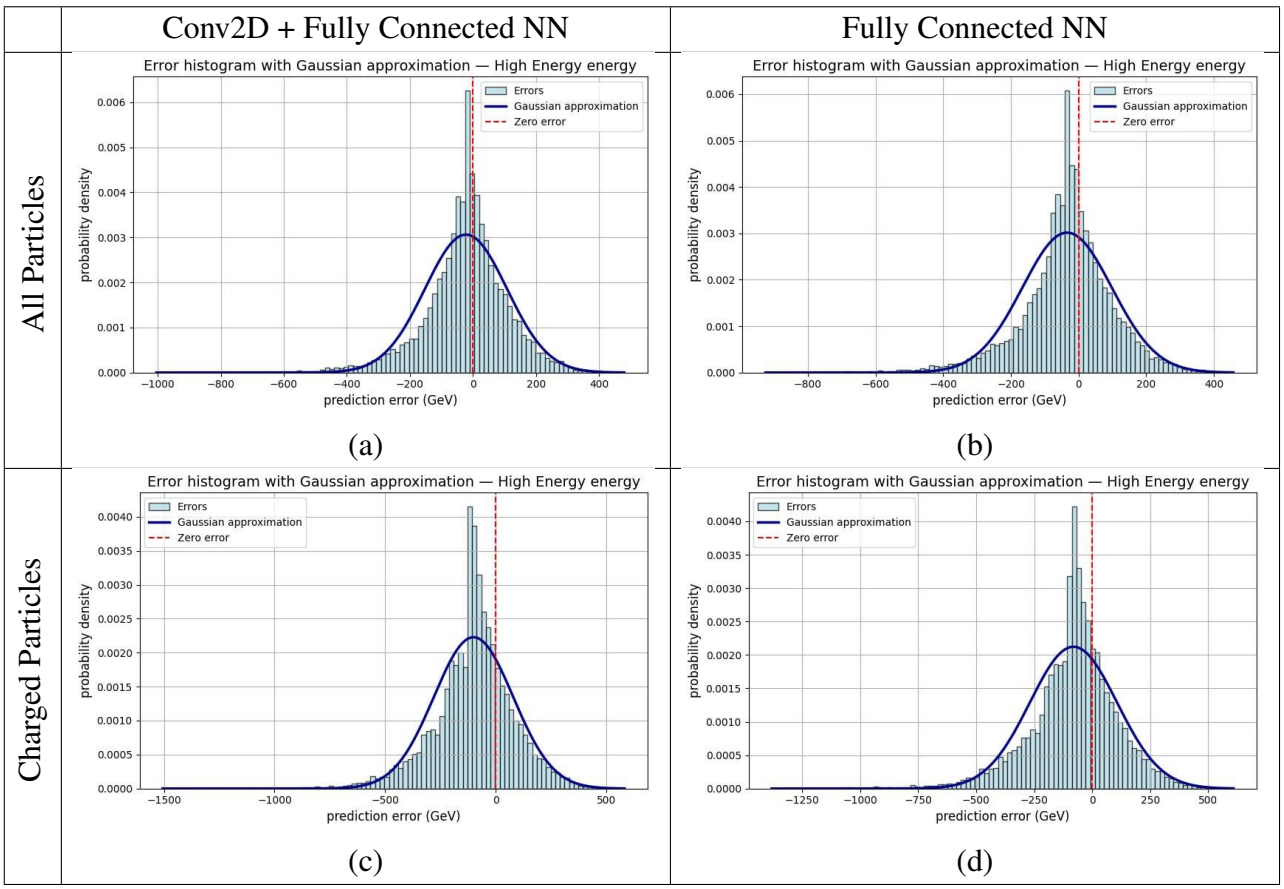


Fig. 17: Histogram of the distribution of the prediction error with Gaussian approximation (High-energy collisions): a – all particles and hybrid NN, b – all particles and fully connected NN, c – charged particles and hybrid NN, d – charged particles and fully connected NN.

The distributions of the prediction error were analysed for three energy ranges: low, medium and high. The obtained histograms (Fig. 13, Fig. 15, Fig. 17) are approximated by a Gaussian, which makes it possible to evaluate the symmetry, shape and shift of the error in each energy interval.

In the region of medium energies, the histograms in all the cases considered show the most symmetric error distribution, closest to the normal one. This indicates that the model is well trained on the data from this range, and also that it is able to stably and fairly accurately estimate the energy released during a nuclear collision in this range.

In the region of low and high energies, the error distribution clearly cannot be described by a Gaussian distribution. A clear asymmetry and broadening of the tails of the distributions is observed. The distributions are shifted: for low energies in the positive direction, and for high energies in the negative one.

Such behaviour may be related to the fact that at the edges of the energy spectrum the predictions of the model are limited by physical boundaries: the energy released during the collision cannot be less than zero (which limits the left "tail" in the region of low energies) and cannot be greater than $\sqrt{s_{total}} = 2168\ GeV$ (which limits the right "tail" in the region of high energies). Such constraints are not taken into account by the model during training, but at the same time they affect the final result: at the edges of the range the model loses the ability to approximate the values of the energy released during the collision, especially when the values are numerically close to the values at the edges of the boundary conditions.

For a deeper analysis of the shape of the prediction error distribution, attempts were made to approximate

the histograms using not only the Gaussian distribution, but also the Lorentz one. For a quantitative evaluation of the quality of the approximation, the adjusted coefficient of determination (Adjusted R^2) was used.

Below are the results for the fully connected NN trained only on charged particles. In the other cases the results are approximately the same.

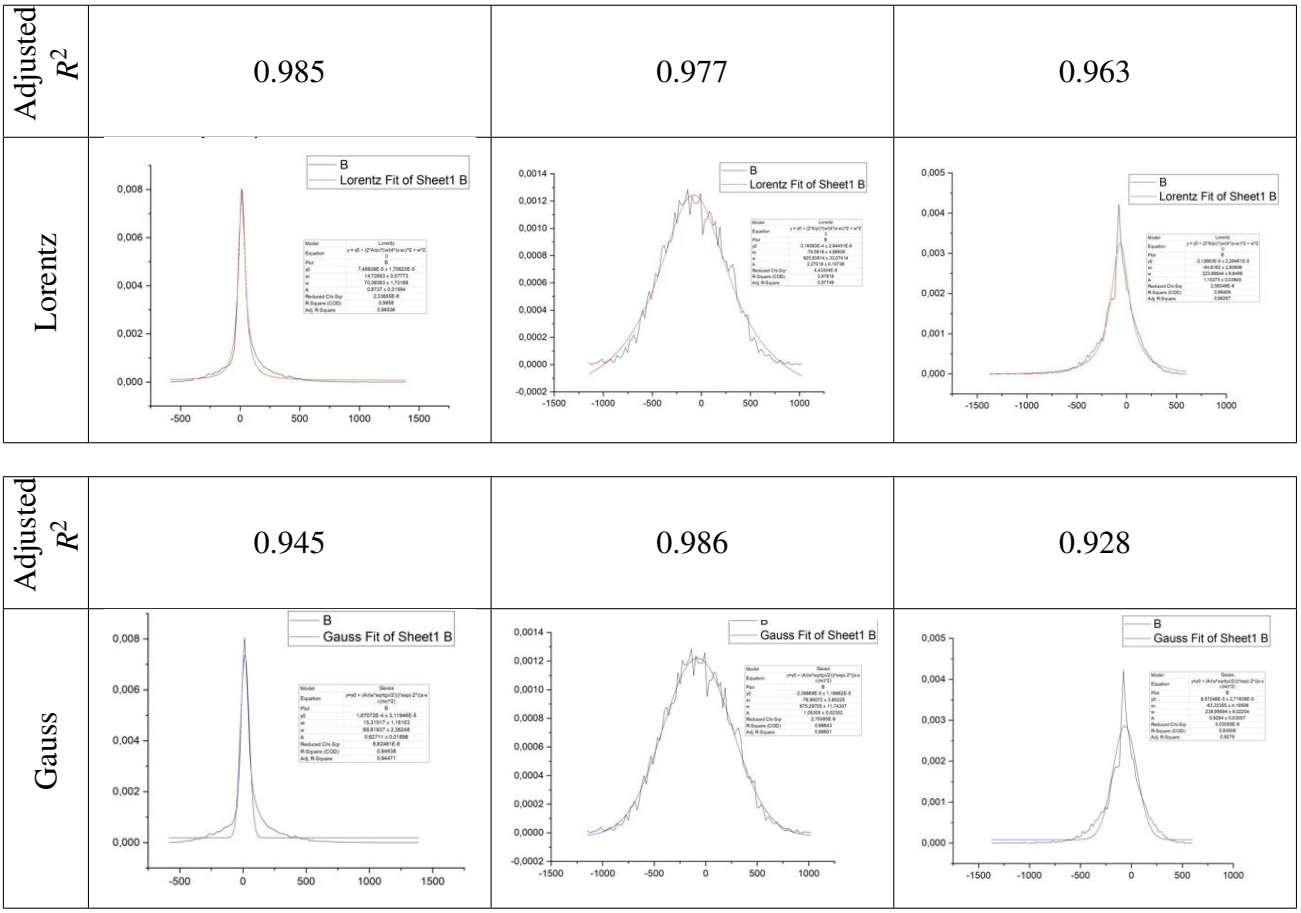


Fig. 17: Histograms of the distribution of the prediction error with approximation by Gaussian and Lorentzian distributions for three energy ranges (from left to right: low, medium, high).

In the region of medium energies, in agreement with the previous observation, the approximation by the normal distribution gives a good result: $Adjusted\ R^2 = 0.986$. In the region of high energies none of the functions gives a good-quality approximation (Lorentz: $Adjusted\ R^2 = 0.963$, Gauss: $Adjusted\ R^2 = 0.928$), which indicates a significant distortion of the shape of the distribution. This may be related to the physical constraint in this energy region, overfitting of the model, a lack of training examples. In the region of low energies the error distribution is well approximated by the Lorentz distribution: $Adjusted\ R^2 = 0.985$. This indicates the presence of pronounced peaks and heavy tails. The fact that in the region of low energies the error distribution is better approximated by the Lorentz function may indicate that the physical constraint on energy is not taken into account during the training of the neural network.

Conclusions and Further Directions of Research

In the course of this work the theoretical dependence $\sqrt{s_{eff}}(b)$ was obtained. The laws of conservation of energy and momentum were checked for the QGSM generator, and also, taking into account the estimate of the CL, the criteria were formulated (the ECL is fulfilled within 1.5%, and the MCL – 14%), by which the energy released during the collision must be determined. However, for the MCL it was found empirically that the plots of the dependence of the collision energy on the impact parameter turn out better within an error of 2%. This may be related to the accuracy of the data. Also, the plots of the dependence of the energy released in the collision on the impact parameter of the collision were constructed: $\sqrt{s_{eff}}(b)$. The obtained dependences coincide with the theoretical ones.

In the second part of the work a function was constructed that maps the signal obtained from the detector on microchannel plates to the energy released in the collision. Two types of model were considered: fully connected and with the addition of a convolutional layer. The neural network was trained first on the whole dataset, and then only on the data containing exclusively charged particles. After carrying out a number of experiments it was found that the model works best if it is trained on the whole dataset, with the addition of a convolutional layer to the architecture. In this case the relative error is $\varepsilon = 6.95\%$, and the root mean squared error is $RMSE = 150.5$. Both values lie within the permissible norm, which means that the model estimates the quantity under study numerically quite accurately.

Also, a deep analysis of three energy ranges was carried out: low, medium and high. Of particular interest are the histograms of the error distribution for all three regions. For the region of medium energies a symmetric shape of the distribution is observed, and the spread of values is fairly small: the error distribution is well approximated by a Gaussian. This indicates the stability of the model in the region of medium energies and its ability to predict the values of the released energy in this region fairly well. For low energies the error distribution is best approximated by the Lorentz distribution function, and for high energies – neither by Lorentz nor by Gauss. This can be explained by physical constraints that are not explicitly taken into account by the model during the training process: the energy released during the collision of two gold nuclei cannot be less than zero and greater than $\sqrt{s_{total}} = 2168$ GeV. Thus, in the range of low energies the "tails" become heavier and the histogram is shifted to the right, and in the region of high energies – to the left.

The results obtained in the course of this work open up possibilities for further improvement of the model. For example, one can try to replace the SiLU activation function with another one that has better properties in the region of high and low energies: this may have a positive effect on the ability of the model to interpret the input data and to build dependencies. In addition, an important step may be preliminary normalisation of the data. Bringing the data to the range of values $[0;1]$ contributes to a more stable and efficient process of training the neural network. It also makes sense to operate not with the energy per two gold nuclei, but per nucleon-nucleon pair. One of the options is the use of other architectures, for example with a larger number of convolutional layers.

Another important direction for further research is the revision of the target functional. In the present work the MSE loss function was applied, which assumes a symmetric error distribution. However, the real error distributions possess asymmetry in the regions where the physical constraint on energies manifests itself. Thus, in the future it is planned to develop an approach that takes into account the asymmetry in the data.

Equipment Used

In the course of this work, to achieve the set tasks, the Python programming language was used, as well as the following libraries:

- Numpy [6]
- Pandas [7]
- Matplotlib [8]
- PyTorch [9]
- Scipy [10]
- Scikit-learn [11]

References

[1] Sudbery A. Quantum Mechanics and the Particles of Nature. – 1989.

[2] Galaktionov K. A., Roudnev V. A., Valiev F. F. Neural Network Approach to Impact Parameter Estimation in High-Energy Collisions Using Microchannel Plate Detector Data //Moscow University Physics Bulletin. – 2023. – Vol. 78. – No. Suppl 1. – P. S52-S58.

[3] Mallick N. et al. Estimating elliptic flow coefficient in heavy ion collisions using deep learning //Physical Review D. – 2022. – Vol. 105. – No. 11. – P. 114022.

[4] Hirvonen H., Eskola K. J., Niemi H. Deep learning for flow observables in ultrarelativistic heavy-ion collisions //Physical Review C. – 2023. – Vol. 108. – No. 3. – P. 034905.

[5] NICA [Electronic resource] – URL: `https://nica.jinr.ru/ru/`

[6] Numpy [Electronic resource] – URL: `https://numpy.org/`

[7] Pandas [Electronic resource] – URL: `https://pandas.pydata.org/`

[8] Matplotlib [Electronic resource] – URL: `https://matplotlib.org/`

[9] PyTorch [Electronic resource] – URL: `https://pytorch.org/`

[10] Scipy [Electronic resource] – URL: `https://scipy.org/`

[11] Scikit-learn [Electronic resource] – URL: `https://scikit-learn.org/`

Appendix 1

Let us consider the derivation of the dependence $\sqrt{s_{eff}}(b)$:

For the collision of two spherical particles the following relation holds for the area of intersection:

$$Area \sim R^2 - b^2 = R^2 \left(1 - \frac{b^2}{R^2}\right)$$

here *Area* is the area of intersection, R is the radius of the spherical particle, b is the impact parameter

The number of particles participating in the interaction:

$$N_{part} \sim \rho_{nucleon} \cdot Volume; \qquad Area \sim Volume,$$

here N_{part} is the number of particles participating in the interaction, $\rho_{nucleon}$ is the nucleon density, $Volume$ is the volume. Then we obtain the relation:

$$N_{part} \sim Area$$

The number of binary nucleon-nucleon interactions is determined through the total number of interacting particles as follows:

$$N_{coll} \sim N_{part}^{\alpha},$$

here $\alpha(b) \in [1;2]$ is some function for which $\alpha|_{b=0} = 2$ and $\alpha|_{b=R} = 1$. Often, to describe the number of interactions inside the interval, $\alpha = 1.5$ is taken.

The effective energy depends on the number of binary nucleon-nucleon collisions and on the energy per nucleon-nucleon pair:

$$\sqrt{s_{eff}} \sim N_{coll}\sqrt{s_{NN}},$$

Here $\sqrt{s_{eff}}$ is the energy in the centre-of-mass system of the particles participating in the interaction.

Thus, combining all the above-mentioned relations we obtain:

$$\sqrt{s_{eff}} \sim N_{part}^{\alpha}\sqrt{s_{NN}} \sim (R^2 - b^2)^{\alpha}\sqrt{s_{NN}}$$

Appendix 2

```
import math
import matplotlib.pyplot as plt
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
import scipy as sp
import seaborn as sns
import rarfile
from scipy.stats import norm
import pandas as pd
Z = 79
A = 197
C = 29.9792 # (cm/ns)
COMP_EN = 11.006
Rel_Err = 0.05 #Relative error (instead of eps = 1e-8)
# define detector geometry

D1 = 5      #inner hole (cm)
D2 = 50     #outer hole (cm)
num_rings = 6 #number of radial rings
num_sectors = 10 #number of sectors(angular divisions)
R1 = D1/2       #inner radius (cm)
R2 = D2/2       #outer radius (cm)
z_detector1 = -400 #distance from the collision point (cm) - 1st detector
z_detector2 = 400   #distance from the collision point (cm) - 2st detector
z_detector = z_detector1 = z_detector2

# define ring and sector boundaries

ring_edges = np.linspace(R1, R2, num_rings + 1)
sector_edges = np.linspace(0, 2 * math.pi, num_sectors + 1)
print(ring_edges)
print(sector_edges)
# particle trajectory simulation


# a function that projects the trajectory of a particle
# and return (x,y) and time of flight (tof)

def project_to_detector(p_x, p_y, p_z, Mass, Z_detector):

    if p_z == 0:
        print("will never cross")        # particle will never touch the detector

    time_parameter = Z_detector / p_z
    x_hit = time_parameter * p_x
    y_hit = time_parameter * p_y
    C = 29.9792 # (cm/ns)
    p = np.sqrt(p_x**2 + p_y**2 + p_z**2)
    energy = np.sqrt(p**2 + Mass**2)
    velocity = (p / energy) * C      # particle velocity in (cm/ns)
    tof = abs(Z_detector) / velocity # time of flight in nanoseconds
    t0 = abs(Z_detector) / C # our shift in time -> the time it takes for the light to reach
```

```
    the detector
    #print('Time = ',tof,'s; shift = ', t0, 's.')
    Tof = 1 / (tof - t0)
    return x_hit, y_hit, Tof
# particle trajectory simulation
# next step - define the pad ; (x,y) - its x_hit and y_hit
# try to determine which pad particle hits. pad = (ring; sector)

def find_pad(x, y):
    # conversion to polar coordinates
    r = np.sqrt(x**2 + y**2)
    fi = np.arctan2(y, x) #returns the angle between [-pi; pi]

    # fi : [0; 2pi]
    if fi < 0:
        fi += 2 * np.pi

    ring_num = np.digitize([r], ring_edges) - 1
    sector_num = np.digitize([fi], sector_edges)
    # checking if the particle hit the detector
    if ring_num[0] <= 0 or ring_num[0] > num_rings:
        return None
    else:
        return [ring_num[0].item(), sector_num[0].item()]
def create_feature_vector(filtered_pads_tof, pz_values, max_particles = 120):

    # create a table filed with zeros
    feature_matrix = np.zeros((3, max_particles))
    #print(feature_matrix)
    # pads and tof extraction
    pads = [pad for pad, _ in filtered_pads_tof] # pads extraction
    tof_s = [tof for _, tof in filtered_pads_tof] # tof extraction
    # number of particles restriction
    num_particles = min(len(tof_s), max_particles)
    #print(num_particles)
    for num in range(num_particles):

        # tof, ring, sector, pz extraction
        tof = tof_s[num]
        #ring, sector = pads[num]
        pz = pz_values[num]

        # detector def
        if pz > 0:
            detector_id = 1
        else:
            detector_id = -1

        # encode pad like 1 number

        pad_encoded = int(''.join(map(str, pads[num])))
        #print(pad_encoded)

            # make a vector
        feature_matrix[0][num] = tof
        feature_matrix[1][num] = pad_encoded
        feature_matrix[2][num] = detector_id
    feature_vector = feature_matrix.T.flatten()
```

```
    return feature_vector
def filter_min_tof(pads, tof_s):

        # create a dictionary to store the min tof for each pad: key -> pad; value -> tof
    pad_to_tof = {}
        # iterate throught all pads and their tof values
    zipped = zip(pads, tof_s)
        #print(zipped)
    for i_pad, j_tof in zipped:

            pad_tuple = tuple(i_pad)    # convert list -> tuple, because dict use cant use
    lists in keys
            # if pad is already in the dict => update if new tof is smaller
            if pad_tuple in pad_to_tof:
                pad_to_tof[pad_tuple] = min(pad_to_tof[pad_tuple], j_tof)
            else:
                pad_to_tof[pad_tuple] = j_tof # else add new pad to dict with its tof


        # convert the dict into a list of lists
    filtered_pads_tof = [[list(pad), tof] for pad, tof in pad_to_tof.items()]


    return filtered_pads_tof
all_features = [] # final vector with all features (independent)
all_energies = [] # final vector with energy per collision (dependent)
files = ['f1.txt', 'f2.txt', 'f3.txt', 'f4.txt', 'f5.txt', 'f6.txt', 'f7.txt', 'f8.txt', 'f9.
    txt', 'f10.txt']

# Path to the archive
rar_path = "C:\\VKR\\pythonProject1\\sourceQGSM_AuAu_11GeV_200k.rar"

# Opening the archive
for i in range(1, 101):
    file_index = f"D:\\200k\\data_{i:d}.dat"

    with open(file_index, 'r') as f:
        lines1 = f.readlines()

    i = 5
    #b_need = []
    pz_values = []
    #energies = []
    #all_events_pads = []
    #all_events_tof_s = []
    #pz_all = []
    while i < len(lines1):
        l = lines1[i]
        n = [float(num) for num in l.split()]
        n_part = int(n[1])
        #b = n[2]
        #bx = n[3]
        #by = n[4]
        #print(f' n = {n_part}')
        i += 1
        prev_i = i
        tot_en = 0 # total energy of all nucleons
```

```
        all_pads = []
        tof_s = []
        pads = []

        while i < prev_i + n_part:
            l = lines1[i]
            params = [float(num) for num in l.split()]
            #type = int(params[4])
            mass = params[10]
            charge = params[0]
            px = abs(params[5])
            py = abs(params[6])
            pz = abs(params[7])
            condition1 = math.isclose(px, pz, abs_tol=0.9 * pz)
            condition2 = math.isclose(py, pz, abs_tol=0.9 * pz)
            #(type == 2212 or type == 2112)
            if (mass >= 9.38200E-01) and (not condition1) and (not condition2):
                energy_csm = (params[5]**2 + params[6]**2 + params[7]**2 + mass**2)**(1/2)

                #print(energy_csm, ' ', COMP_EN)
                if (math.isclose(energy_csm, COMP_EN, rel_tol = COMP_EN * Rel_Err)):
                    #print(b, energy_csm)
                    t = 0
                    #print(l)
                else:
                    tot_en += energy_csm
                    # pz_values.append(pz)
                    x, y, t = project_to_detector(px, py, pz, mass, z_detector)
                    #print(t)
                    pad = find_pad(x, y)
                    all_pads.append(pad)
                    # deleting [-100;100] - if particles didnt hit the detector
                    pads = [pad for pad in all_pads if pad is not None]
                    pz_values.append(pz)
                    tof_s.append(t.item())
                    #print(f'charge: {charge} and i = {i}')

            i += 1

        all_energies.append(tot_en)
        filtered = filter_min_tof(pads, tof_s)

        feature_vec = create_feature_vector(filtered, pz_values)
        all_features.append(feature_vec)
        #all_events_pads.append(pads)
        #all_events_tof_s.append(tof_s)
        #pz_all.append(pz_values)
with open('all_features.txt', 'w') as f:
    for i in range(len(all_features)):
        f.write(str(all_features[i]))
feature_matrixx = np.ones((len(all_features), 360))

for i in range(len(all_features)):
    feature_matrixx[i][:] = all_features[i]

print(feature_matrixx.shape)
# makes train and test sets
```

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(feature_matrixx, all_energies, test_size
    =0.2, random_state=42)
# step 2 - create our own DataSet class
# features - list of feature vectors, one for each event <=> dim = [N, 3*120] N - number of
    events
# energies - list of energies in every event <=> dim = N

class QGSMCollisionDataset(Dataset):
    def __init__(self, all_features, all_energies):
        self.X = torch.tensor(all_features, dtype = torch.float32)
        self.y = torch.tensor(all_energies, dtype = torch.float32).unsqueeze(1) # [N, 1]
    def __len__(self):
        return len(self.X)
    def __getitem__(self, idx):
        return self.X[idx], self.y[idx]
# step 3 - neural network architecture
class EnergyRegressor(nn.Module):
    def __init__(self):
        super(EnergyRegressor, self).__init__()
        self.net = nn.Sequential(
            nn.Linear(360, 128),
            nn.SiLU(),
            nn.Linear(128, 64),
            nn.SiLU(),
            nn.Linear(64, 32),
            nn.SiLU(),
            nn.Linear(32, 16),
            nn.SiLU(),
            nn.Linear(16, 1)
        )


    def forward(self, x):
        f_out = self.net(x)
        return f_out
class EnergyRegressor(nn.Module):
    def __init__(self):
        super(EnergyRegressor, self).__init__()

        self.conv = nn.Sequential(
            nn.Conv2d(in_channels=1, out_channels=8, kernel_size=(6, 3)),
            nn.ReLU()
        )


        self.fc = nn.Sequential(
            nn.Flatten(),
            nn.Linear(8 * 115 * 1, 128),
            nn.SiLU(),
            nn.Linear(128, 64),
            nn.SiLU(),
            nn.Linear(64, 32),
            nn.SiLU(),
            nn.Linear(32, 16),
            nn.SiLU(),
            nn.Linear(16, 1)
        )
```

```
    def forward(self, x):
        x = x.view(-1, 1, 120, 3)
        x = self.conv(x)
        x = self.fc(x)
        return x
# step 4 - training our model

def train_model(X_train, y_train, X_test, y_test, epochs = 1500, batch_size = 3000,
    learning_rate = 0.001, patience = 100):

    dataset = QGSMCollisionDataset(X_train, y_train)
    loader = DataLoader(dataset, batch_size = batch_size, shuffle = True) # split on batches
    and mix them
    X_test = torch.tensor(X_test, dtype=torch.float32)
    y_test = torch.tensor(y_test, dtype=torch.float32).unsqueeze(1)

    model = EnergyRegressor()
    criterion = nn.MSELoss()
    optimizer = optim.Adam(model.parameters(), lr = learning_rate, weight_decay=1e-5)

    train_losses = []
    test_losses = []

    best_val_loss = float('inf')    # least validation error
    best_model_state = None # best weight models
    patience_counter = 0    # add one if the error does not decrease

    print("Minimum:", torch.min(torch.tensor(y_train, dtype=torch.float32)).item())
    print("Maximum:", torch.max(torch.tensor(y_train, dtype=torch.float32)).item())
    print("Mean:", torch.mean(torch.tensor(y_train, dtype=torch.float32)).item())

    for epoch in range(epochs):
        model.train()
        running_loss = 0.0
        for batch_X, batch_y in loader:
            #print('X.shape = ',batch_X.shape, "Y.shape = ", batch_y.shape)
            optimizer.zero_grad()       # reset gradients -> 0
            outputs = model(batch_X)    # make a prediction
            loss = criterion(outputs, batch_y) # calculate out loss
            loss.backward()     # calculate gradients
            optimizer.step()    # update weights
            running_loss += loss.item()

        avg_train_loss = running_loss / len(loader) # average loss for 1 epoch
        train_losses.append(avg_train_loss) # for visualization

        # error estimation on the test sample
        model.eval()
        with torch.no_grad():
            test_preds = model(X_test)
            test_loss = criterion(test_preds, y_test).item()
        test_losses.append(test_loss)

        # early stopping
        if test_loss < best_val_loss:
            best_val_loss = test_loss
            best_model_state = model.state_dict()
```

```
            patience_counter = 0
        else:
            patience_counter += 1
            if patience_counter >= patience:
                print(f"early stopping triggered at epoch {epoch}")
                break

        if epoch % 100 == 0:
            print(f"running loss: {running_loss}", f"epoch: {epoch}")

    model.load_state_dict(best_model_state)

    # visualization
    plt.figure(figsize=(10, 5))
    plt.loglog(train_losses, color = 'red', label = 'Loss(TRAIN)')
    plt.loglog(test_losses, color = 'blue', label = 'Loss(TEST)')
    plt.title('Train VS Test Loss Evaluation')
    plt.xlabel('epoch')
    plt.ylabel('MSE - our loss')
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.show()


    return model
r = train_model(X_train, y_train, X_test, y_test)
print(r)
# predicted and true energy values
X_test = torch.tensor(X_test, dtype=torch.float32)
y_test = torch.tensor(y_test, dtype=torch.float32).unsqueeze(1)

r.eval()
with torch.no_grad():
    y_true = y_test.squeeze().numpy()
    y_pred = r(X_test).squeeze().detach().numpy()
min_energy = y_true.min()
max_energy = y_true.max()
step = (max_energy - min_energy) / 3
d1 = min_energy + step
d2 = d1 + step
d3 = d2 + step
print(f'Step: {step}')
print(f'Min Energy: {min_energy}')
print(f'Max Energy: {max_energy}')
print(f'd1 = {d1}')
print(f'd2 = {d2}')
print(f'd3 = {d3}')
from sklearn.metrics import mean_squared_error
# metric intro on the test set
rmse = np.sqrt(mean_squared_error(y_true, y_pred))
print(f"RMSE: {rmse:.5f} GeV")
def plot_error_hist(errors, label):
    #errors_np = errors.numpy()
    mu = np.mean(errors)
    sigma = np.std(errors)
    x = np.linspace(min(errors), max(errors), 1000)
    gaussian_curve = norm.pdf(x, mu, sigma)
```

```
    plt.figure(figsize=(8, 5))
    plt.hist(errors, bins=100, density=True, color='lightblue', edgecolor='black', alpha=0.7,
    label='Errors')


    plt.plot(x, gaussian_curve, color='darkblue', linewidth=2.5, label='Gaussian approximation
    ')
    plt.axvline(0, color='red', linestyle='--', label='Zero error')
    plt.title(f"Error histogram with Gaussian approximation -- {label} energy", fontsize=14)
    plt.xlabel("prediction error (GeV)", fontsize=12)
    plt.ylabel("probability density", fontsize=12)
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.show()
    #import pandas as pd

    #counts, bin_edges = np.histogram(errors, bins=100, density=True)
    #hist_data = pd.DataFrame({
    #    'Bin_Center': (bin_edges[:-1] + bin_edges[1:]) / 2, # Interval centers
    #    'Counts': counts
    #})


    hist_data.to_csv('histogram_data.csv', index=False)

    #print("bin edges x:", bin_edges)
    #print("counts y:", counts)
# separation of energies by ranges (low, medium, high)

low_mask = y_true < d1
mid_mask = (y_true >= d1) & (y_true <= d2)
high_mask = y_true > d2

energy_ranges = {
    'Low Energy': low_mask,
    'Mid Energy': mid_mask,
    'High Energy': high_mask
}

for title, mask in energy_ranges.items():
    y_t = y_true[mask]
    y_p = y_pred[mask]
    errors = y_p - y_t

    plt.figure(figsize=(12, 5))

    # Scatter Plot
    plt.subplot(1, 2, 1)
    plt.scatter(y_t, y_p, s=2, alpha=0.3)
    plt.plot([y_t.min(), y_t.max()], [y_t.min(), y_t.max()], color='orange', label='Perfect
    prediction')
    plt.xlabel("True Energy, GeV")
    plt.ylabel("Predicted Energy, GeV")
    plt.title(f"{title} - Scatter")
    plt.legend()

    # Density + Contour
```

```
    plt.subplot(1, 2, 2)
    sns.kdeplot(x=y_t, y=y_p, fill=True, cmap="Purples", levels=20, thresh=0.01, linewidths
    =2.5, linestyles='solid')
    plt.plot([y_t.min(), y_t.max()], [y_t.min(), y_t.max()], color='orange', label='Perfect
    prediction')
    plt.xlabel("True Energy")
    plt.ylabel("Predicted Energy")
    plt.title("Density Plot")
    plt.legend()

    # Histogram

    plot_error_hist(errors, title)


plt.tight_layout()
plt.show
```